

Program Analysis 2025-26

Lecture #14

Abstract numerical domains



UNIVERSITÀ
DI PISA



Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation

Antoine Miné

► To cite this version:

Antoine Miné. Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation. Foundations and Trends in Programming Languages, Now Publishers, 2017, 4 (3-4), pp.120-372. 10.1561/25000000034 . hal-01657536

HAL Id: hal-01657536

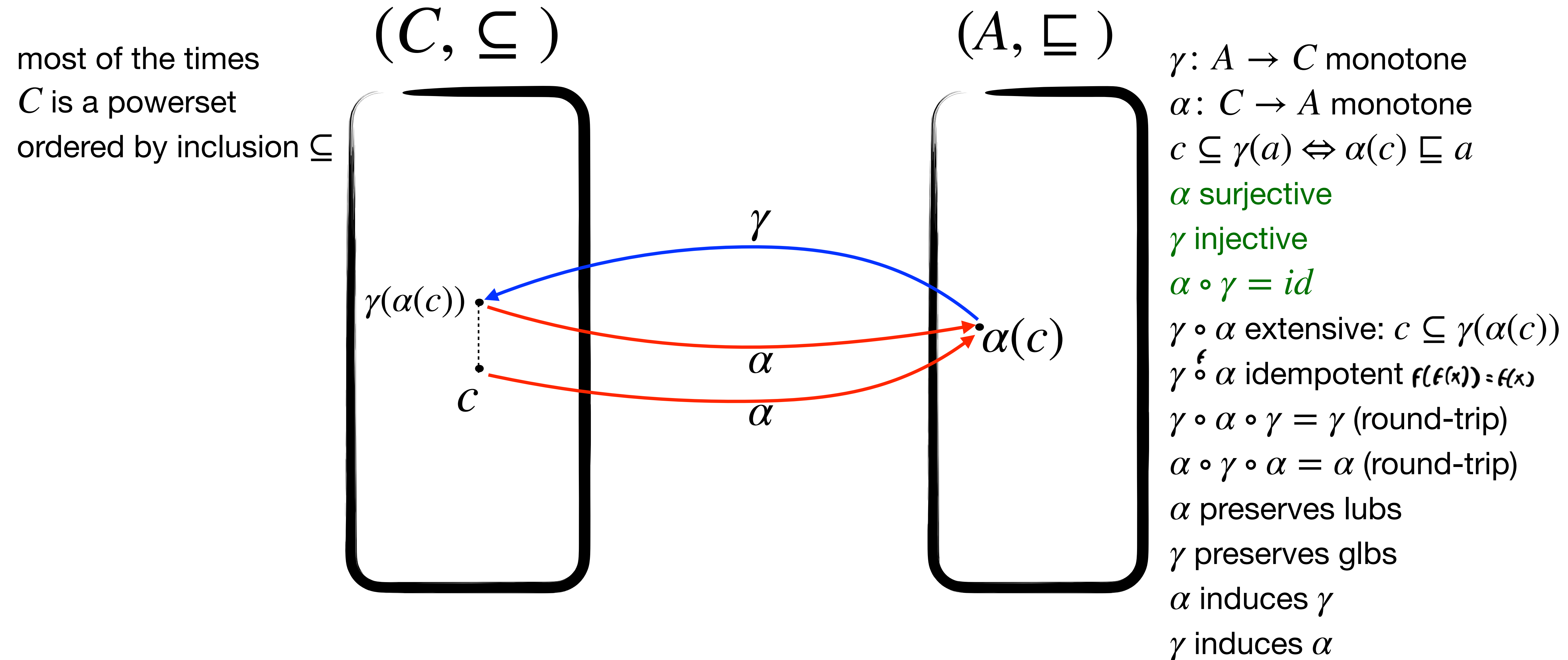
<https://hal.sorbonne-universite.fr/hal-01657536>

Submitted on 1 May 2018

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers. L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

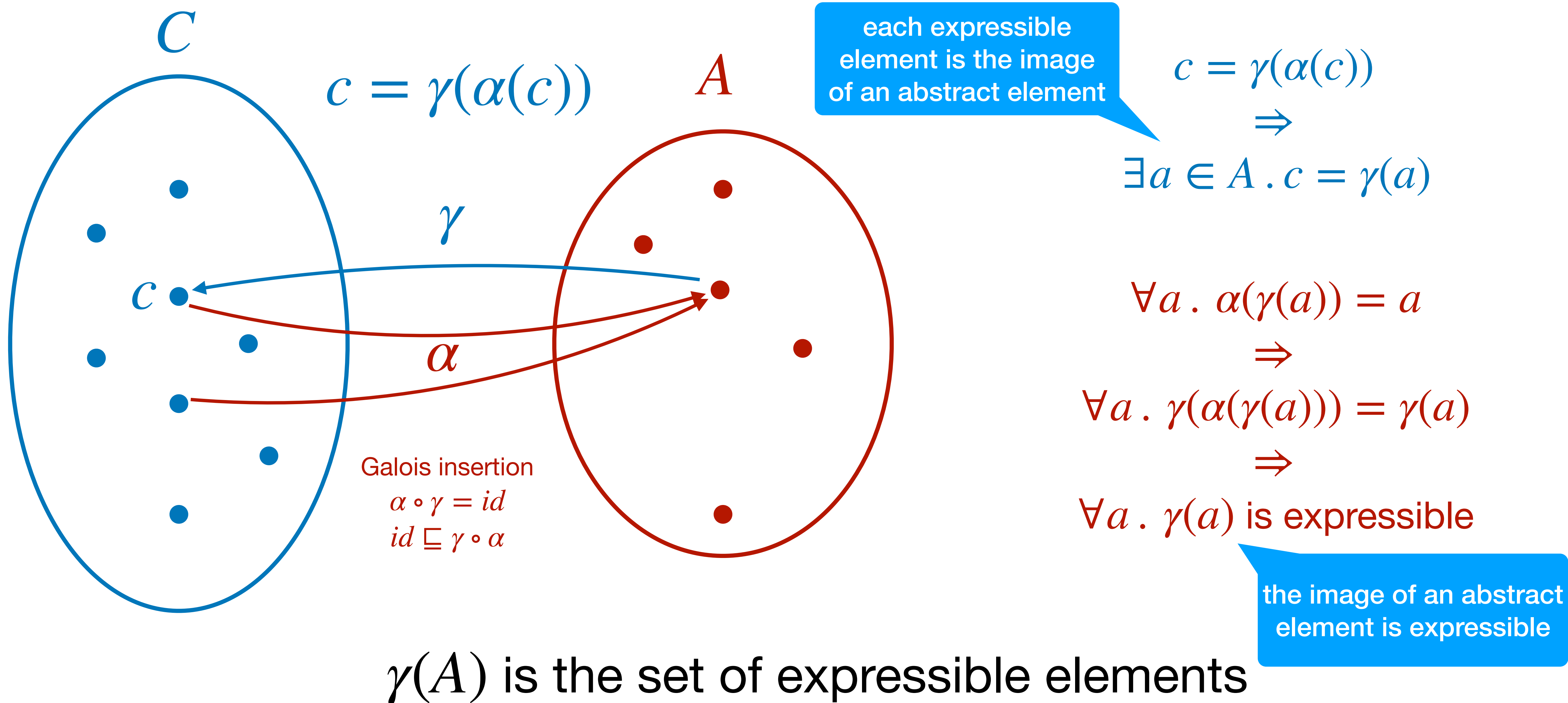
Chapter 2: Elements of Abstract Interpretation

Focus on Galois insertions

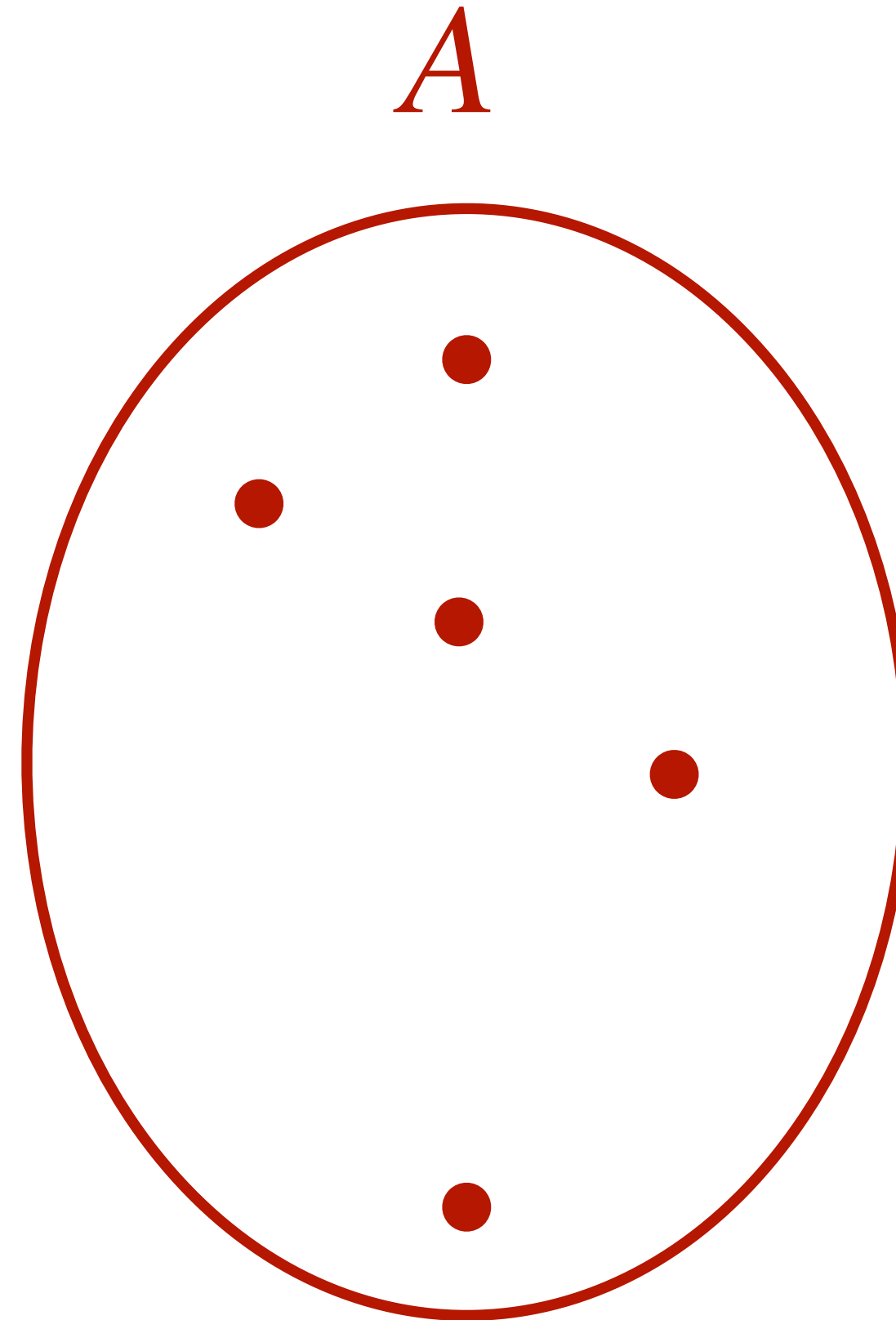
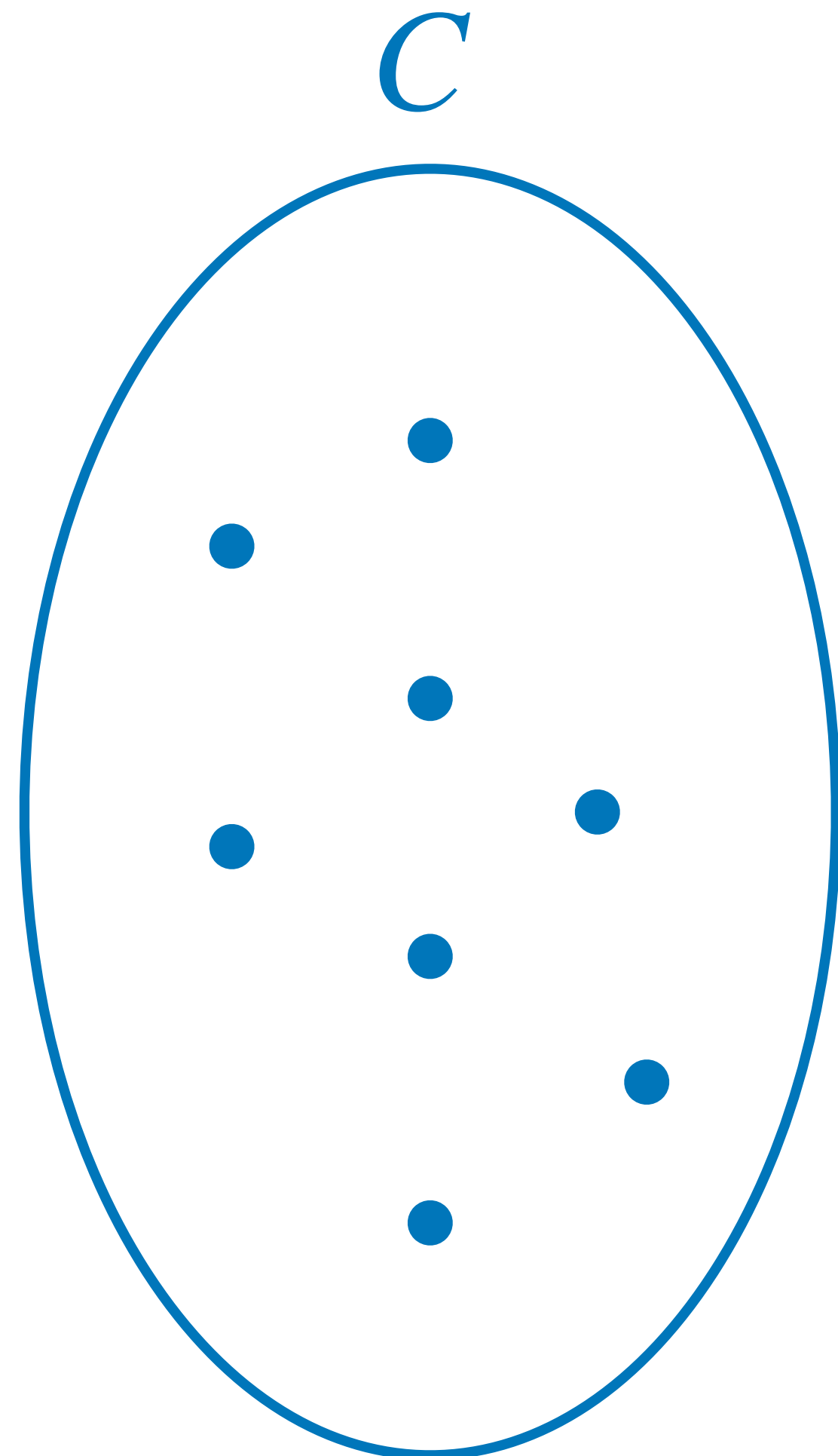


Addendum: closure operators

Expressible elements



Galois insertion as closures



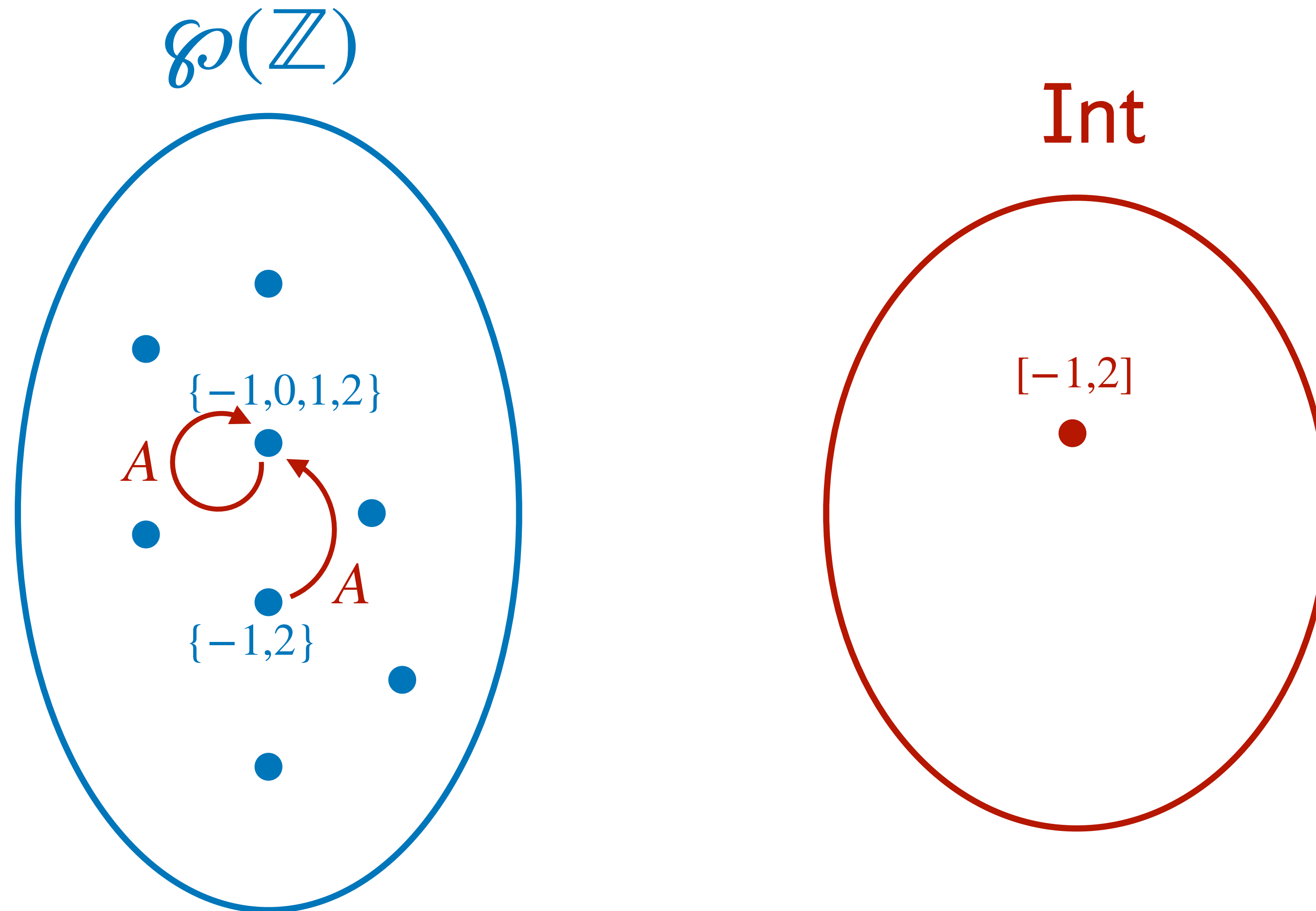
The abstract domain
can just be seen as
a subset of the concrete domain

We write $A(c)$
as a shorthand for $\gamma(\alpha(c))$

$A(C)$ is the set of
expressible elements

Since $A(A(c)) = A(c)$ the map $A : C \rightarrow C$ is a **closure** operator
(monotone, extensive, idempotent)

Example

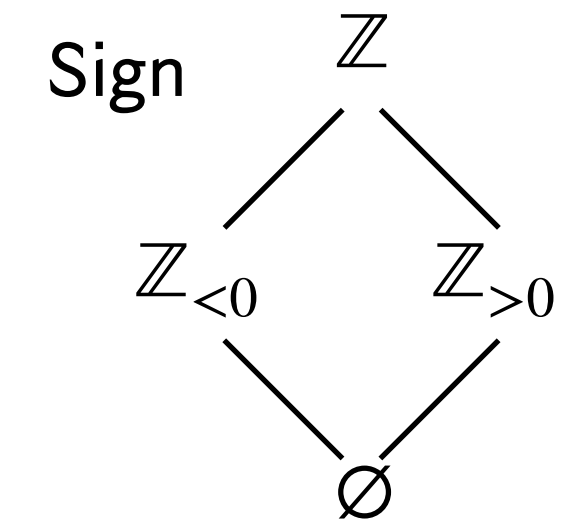


No need of symbolic representations

Examples

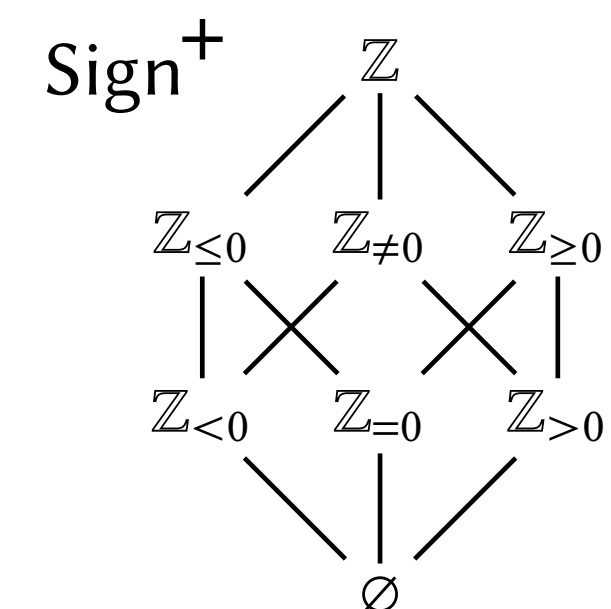
$$\text{Int}(\{2,4,6,\dots\}) = \{2,3,4,5,6,\dots\} = [2,\infty]$$

$$\text{Sign}(\{2,4,6,\dots\}) = \{1,2,3,4,5,6,\dots\} = \mathbb{Z}_{>0}$$



$$\text{Int}(\{0,2,4,6,\dots\}) = \{0,1,2,3,4,5,6,\dots\} = [0,\infty]$$

$$\text{Sign}(\{0,2,4,6,\dots\}) = \{\dots, -1, 0, 1, 2, 3, 4, 5, 6,\dots\} = \mathbb{Z}$$



$$\text{Sign}^+(\{0,2,4,6,\dots\}) = \{0,1,2,3,4,5,6,\dots\} = \mathbb{Z}_{\geq 0}$$

Sound abstract operators and BCAs

Operator approximation

For every concrete operator $F : C \rightarrow C$
we must define a corresponding operator $F^\# : A \rightarrow A$ in the abstract domain

From the concretization map γ we derive the notion of sound / exact abstraction

$F^\#$ is a **sound abstraction** of F if $\forall a \in A :$ $F(\gamma(a)) \subseteq \gamma(F^\#(a))$

Operator approximation

For every concrete operator $F : C \rightarrow C$
we must define a corresponding operator $F^\# : A \rightarrow A$ in the abstract domain

From the concretization map γ we derive the notion of sound / exact abstraction

$F^\#$ is a **sound abstraction** of F if $\forall a \in A :$ $F(\gamma(a)) \subseteq \gamma(F^\#(a))$
or equivalently (by definition of GC) if $\forall a \in A : \alpha(F(\gamma(a))) \sqsubseteq F^\#(a)$

$$\text{GC: } c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$$

Operator approximation

For every concrete operator $F : C \rightarrow C$
we must define a corresponding operator $F^\# : A \rightarrow A$ in the abstract domain

From the concretization map γ we derive the notion of sound / exact abstraction

$F^\#$ is a **sound abstraction** of F if $\forall a \in A :$ $F(\gamma(a)) \subseteq \gamma(F^\#(a))$
or equivalently (by definition of GC) if $\forall a \in A : \alpha(F(\gamma(a))) \sqsubseteq F^\#(a)$
or equivalently (see next slides) if $\forall c \in C : \alpha(F(c)) \sqsubseteq F^\#(\alpha(c))$

$$\text{GC: } c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$$

Operator approximation

For every concrete operator $F : C \rightarrow C$
we must define a corresponding operator $F^\# : A \rightarrow A$ in the abstract domain

From the concretization map γ we derive the notion of sound / exact abstraction

$F^\#$ is a **sound abstraction** of F if
or equivalently (by definition of GC) if
or equivalently (see next slide) if

$$\begin{aligned} F \circ \gamma &\subseteq \gamma \circ F^\# \\ \alpha \circ F \circ \gamma &\sqsubseteq F^\# \\ \alpha \circ F &\sqsubseteq F^\# \circ \alpha \end{aligned}$$

using functional notation

$$f \leq g \Leftrightarrow \forall x. f(x) \leq g(x)$$

$$\text{GC: } c \leq \gamma(a) \Leftrightarrow \alpha(c) \sqsubseteq a$$

Sound approximation

$$\alpha \circ F \circ \gamma \sqsubseteq F^\# \iff \alpha \circ F \sqsubseteq F^\# \circ \alpha$$

$$\alpha \circ F \circ \gamma \sqsubseteq F^\#$$

(function composition) $\iff$

$$\alpha \circ F \circ \gamma \circ \alpha \sqsubseteq F^\# \circ \alpha$$

($\gamma \circ \alpha$ extensive) $\implies$

$$\alpha \circ F \sqsubseteq F^\# \circ \alpha$$

$$\alpha \circ F \sqsubseteq F^\# \circ \alpha$$

(function composition) $\implies$

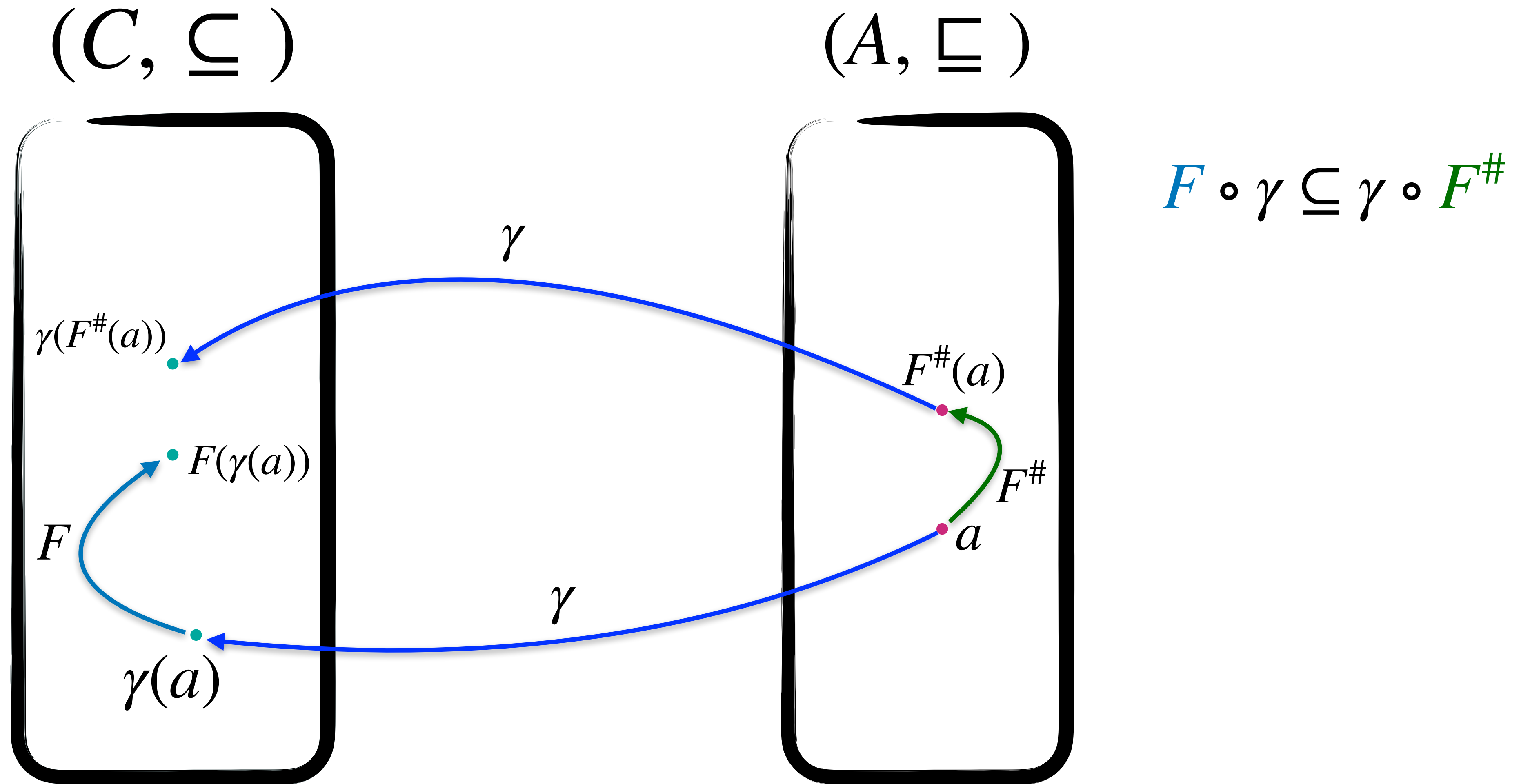
$$\alpha \circ F \circ \gamma \sqsubseteq F^\# \circ \alpha \circ \gamma$$

($\alpha \circ \gamma$ reductive) $\implies$

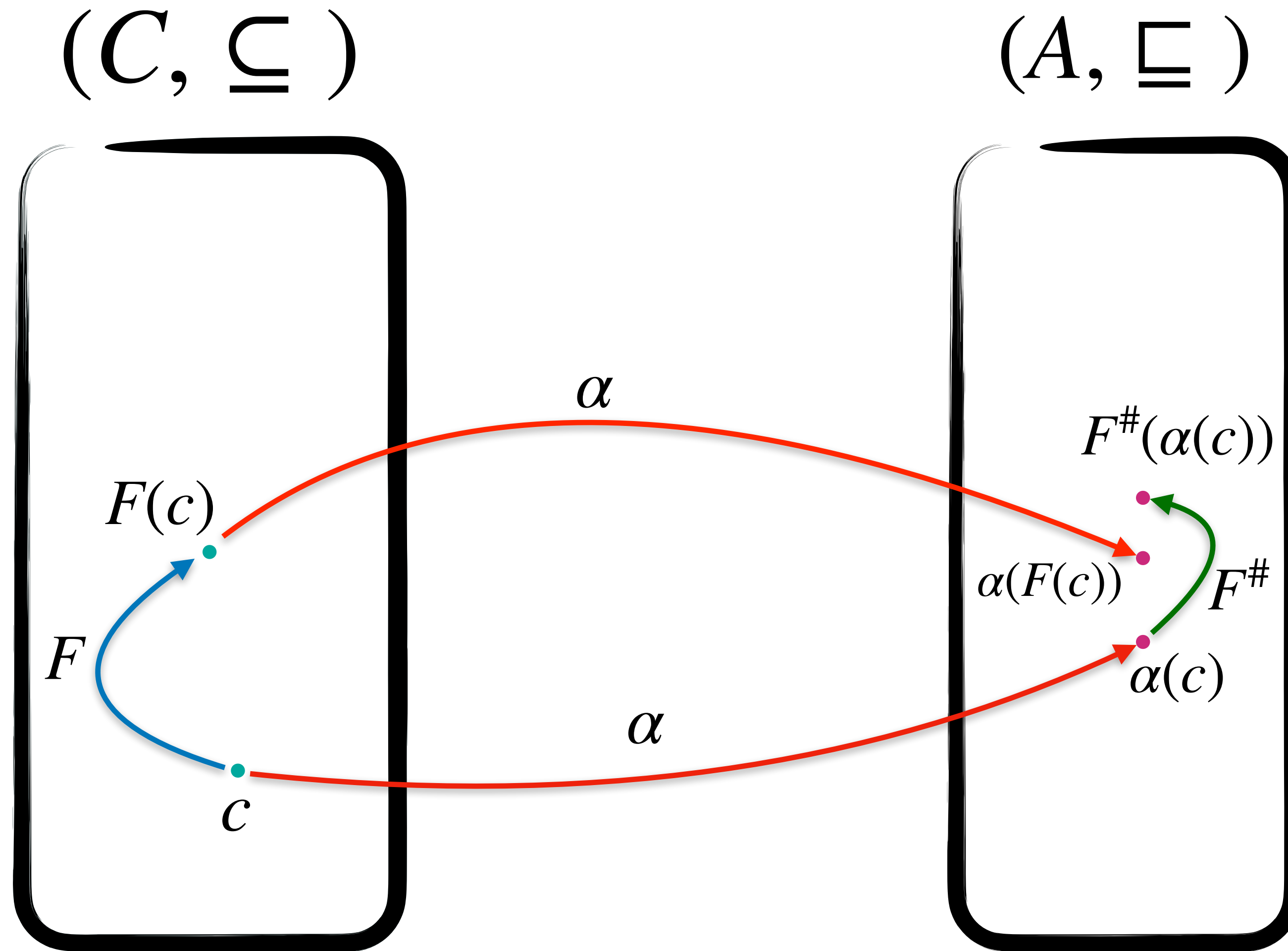
$$\alpha \circ F \circ \gamma \sqsubseteq F^\#$$

any sound approximation $F^\#$ is less precise than $F^A \triangleq \alpha \circ F \circ \gamma$

Sound approximations



Sound approximations

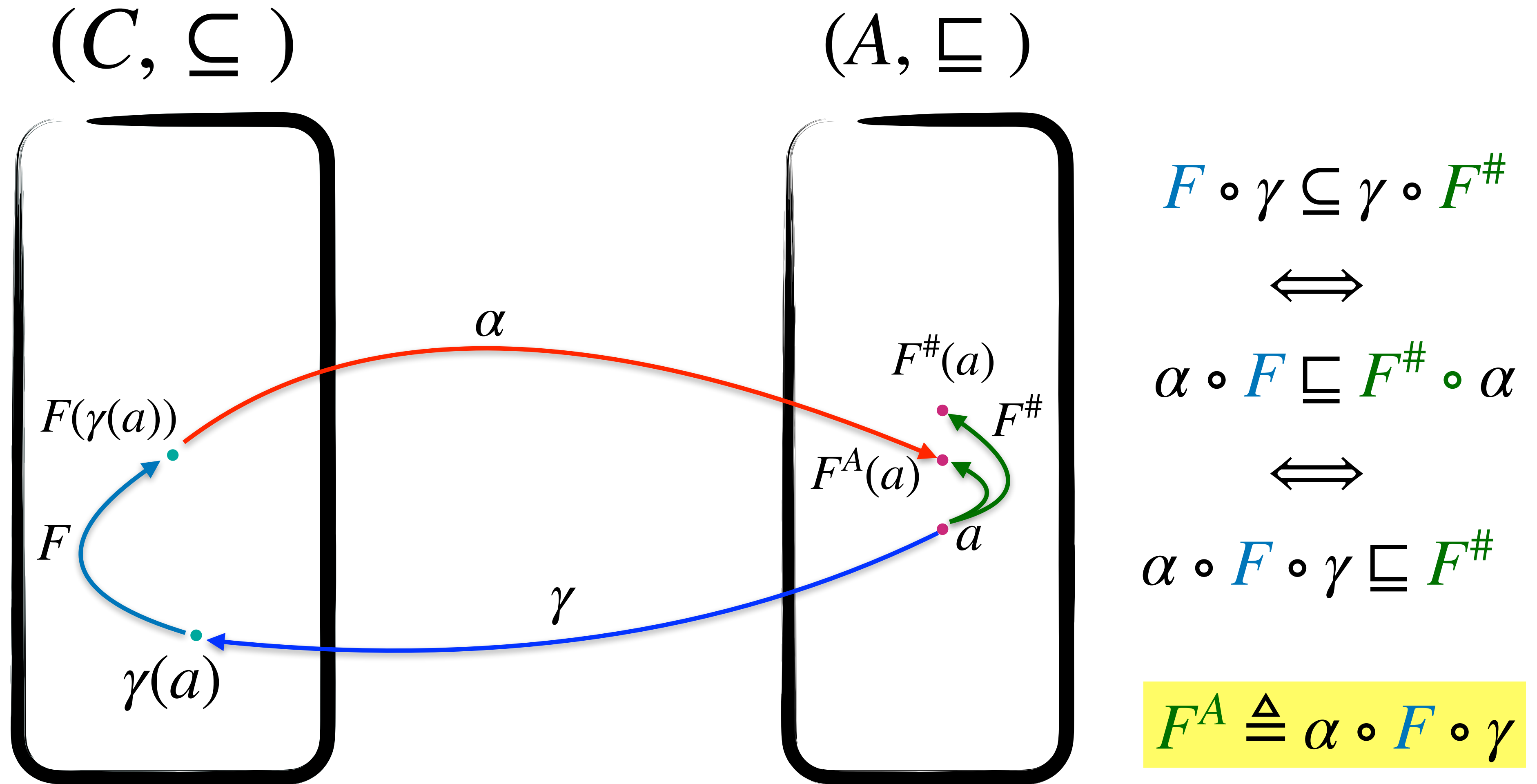


$$F \circ \gamma \subseteq \gamma \circ F^\#$$

$$\iff$$

$$\alpha \circ F \sqsubseteq F^\# \circ \alpha$$

Best correct approximation (bca)



Definition 2.16 (Best operator abstraction). Given a Galois connection $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$ and a concrete operator $f : C \rightarrow C$, the best abstraction of f is given by $\alpha \circ f \circ \gamma$.

Examples

$$F(\gamma(a)) \subseteq \gamma(F^\#(a))$$

↑
sound?

$$\rho_{\alpha(F(\gamma(\cdot)))}$$

BCA?

$$\wp(\mathbb{Z})$$

Int

$$F \triangleq \lambda X. \{x + 1 \mid x \in X\}$$

$$F_1^\# \triangleq \lambda[a, b]. [-\infty, +\infty]$$



$$F_2^\# \triangleq \lambda[a, b]. [a + 1, b + 1]$$



$$F_3^\# \triangleq \lambda[a, b]. [a, b + 1]$$



$$G \triangleq \lambda X. \{2x \mid x \in X\}$$

$$G_1^\# \triangleq \lambda[a, b]. [2a, 2b]$$



$$G_2^\# \triangleq \lambda[a, b]. [a, 2b]$$



Sound operator composition

Theorem. Let $F_1 : C \rightarrow C$ and $F_2 : C \rightarrow C$ and be concrete (monotone) operators and let $F_1^\# : A \rightarrow A$ and $F_2^\# : A \rightarrow A$ some corresponding sound abstractions. Then, $F_2^\# \circ F_1^\#$ is a sound abstraction of $F_2 \circ F_1$.

Proof. $(F_2 \circ F_1) \circ \gamma \subseteq F_2 \circ \gamma \circ F_1^\# \subseteq \gamma \circ (F_2^\# \circ F_1^\#)$

$$F_1 \circ \gamma \subseteq \gamma \circ F_1^\#$$

$$F_2 \circ \gamma \subseteq \gamma \circ F_2^\#$$

BCA composition?

Remark. Let $F_1 : C \rightarrow C$ and $F_2 : C \rightarrow C$ and be concrete (monotone) operators and let $F_1^A : A \rightarrow A$ and $F_2^A : A \rightarrow A$ their corresponding BCAs. Then, $F_2^A \circ F_1^A$ is a sound abstraction of $F_2 \circ F_1$ but not necessarily its BCA.

$$F_2^A \circ F_1^A = \alpha \circ F_2 \circ \gamma \circ \alpha \circ F_1 \circ \gamma \quad \text{vs.} \quad \alpha \circ (F_2 \circ F_1) \circ \gamma$$

The composition $F_2^A \circ F_1^A$ performs an additional trip $\gamma \circ \alpha$ through the abstract world that represents a potential source of precision loss

Thus, in general: $(F_2 \circ F_1)^A \sqsubseteq F_2^A \circ F_1^A \sqsubseteq F_2^\# \circ F_1^\#$

BCA composition?

$\wp(\mathbb{Z})$	Int	sound?	BCA?
$F_1 \triangleq \lambda X. \{2x \mid x \in X\}$	$F_1^\# \triangleq \lambda[a, b]. [2a, 2b]$	✓	✓
$F_2 \triangleq \lambda X. \{x \in X \mid x \leq 1\}$	$F_2^\# \triangleq \lambda[a, b]. [a, \min(1, b)]$	✓	✓
$(\alpha \circ (F_2 \circ F_1) \circ \gamma) [0, 1]$ $= (\alpha \circ F_2 \circ F_1) \{0, 1\}$ $= (\alpha \circ F_2) \{0, 2\}$ $= \alpha \{0\}$ $= [0, 0]$	$(F_2^\# \circ F_1^\#) [0, 1]$ $= F_2^\# [0, 2]$ $= [0, 1]$		

Abstract domains

General considerations

the selection of an adequate abstract domain is at the foundation of the design of any static analysis tool
(it fixes the set of properties used in the analysis)

the set of properties in an abstract domain is closed under intersection (called a Moore family)

when defining an adjunction,
each adjoint uniquely determines the other

Pirahã domain

Pirahã people

Article [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#)

From Wikipedia, the free encyclopedia

The **Pirahã** (Portuguese pronunciation: [pɨɾaˈhẽ]^[a] are an indigenous people of the Amazon rainforest in Brazil. They are the sole surviving subgroup of the Mura people and are hunter-gatherers. They live mainly on the banks of the Maici River in Humaitá and Manicoré in the state of Amazonas. As of 2024, they numbered 900 individuals.^[3] The name *Pirahã* is an **exonym**; the Pirahã call themselves the *Híaitíhi* or *Hiáitihí*,^[4] roughly translated as "the straight ones".^[5]

The Pirahã speak the **Pirahã language**. They call any other language "crooked head".^[6] Members of the Pirahã can **whistle their language**, which is how Pirahã men communicate when hunting in the jungle.

Culture [\[edit\]](#)

Pirahã

Híaitíhi, Hiáitihí



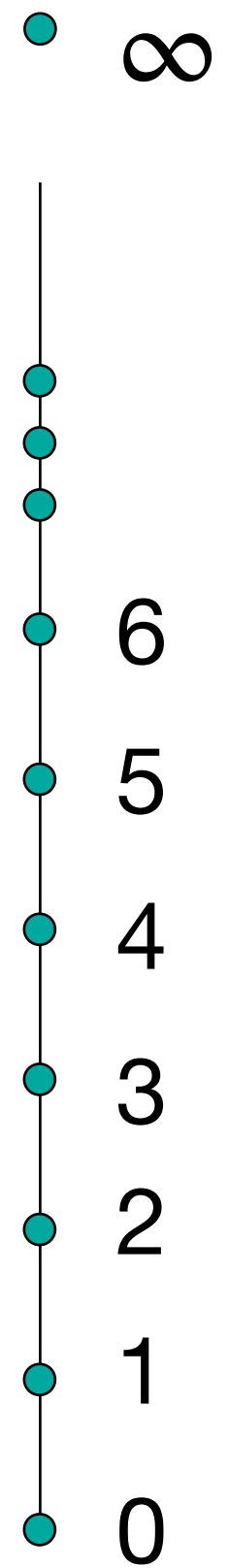
A Pirahã family in 2018

Total population

900 (2024)

The language does not have words for precise numbers, but rather concepts for a small amount and a larger amount.

Pirahã domain

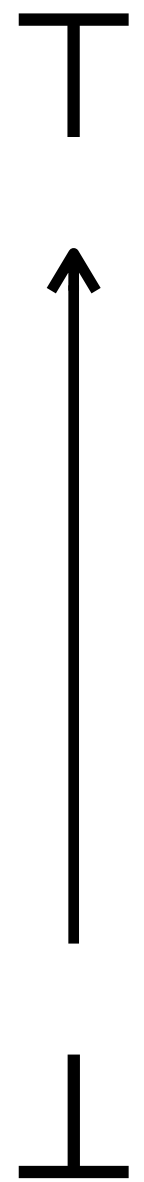


$(\mathbb{N}^\infty, \leq)$

$$\gamma(\top) = \infty$$

$$\gamma(\perp) = 3$$

$$\alpha(n) = \begin{cases} \perp & \text{if } n \leq 3 \\ \top & \text{otherwise} \end{cases}$$



$$(\mathbb{N}^\infty, \leq) \begin{matrix} \xleftarrow{\gamma} \\ \xrightarrow{\alpha} \end{matrix} (A_{\mathbb{P}}, \sqsubseteq)$$

Exercise: sound abstractions

$$\gamma(\top) = \infty$$

$$\gamma(\perp) = 3$$

$+\#$	$\perp$	$\top$
$\perp$	$\top$	(commutative)
$\top$	$\perp$	$\perp$

sound? BCA?

$+\#$	$\perp$	$\top$
$\perp$	$\perp$	(commutative)
$\top$	$\top$	$\top$

sound? BCA?

$+\#$	$\perp$	
$\perp$	$\top$	(commutative)
$\top$	$\top$	$\top$

sound? BCA?

For the curious minded

Numerals and grammatical number [\[edit \]](#)

According to Everett, Pirahã has words for 'one' (*hói*) and 'two' (*hoí*), distinguished only by tone. In his 2005 analysis, however, Everett said that Pirahã has no words for numerals at all, and that *hói* and *hoí* actually mean "small quantity" and "larger quantity".^[14] Michael Frank, Everett, and several others have described two experiments on four Pirahã speakers that were designed to test these two hypotheses.^[2]

In one, ten spools of thread were placed on a table one at a time and the Pirahã were asked how many were there. All four speakers answered in accordance with the hypothesis that the language has words for 'one' and 'two' in this experiment, uniformly using *hói* for one spool, *hoí* for two spools, and a mixture of the second word and 'many' for more than two spools.

The second experiment, however, started with ten spools of thread on the table, and spools were subtracted one at a time. In this experiment, one speaker used *hói* (the word previously supposed to mean 'one') when there were six spools left, and all four speakers used that word consistently when there were as many as three spools left. Though Frank and his colleagues do not attempt to explain their subjects' difference in behavior in these two experiments, they conclude that the two words under investigation "are much more likely to be relative or comparative terms like 'few' or 'fewer' than absolute terms like 'one'".

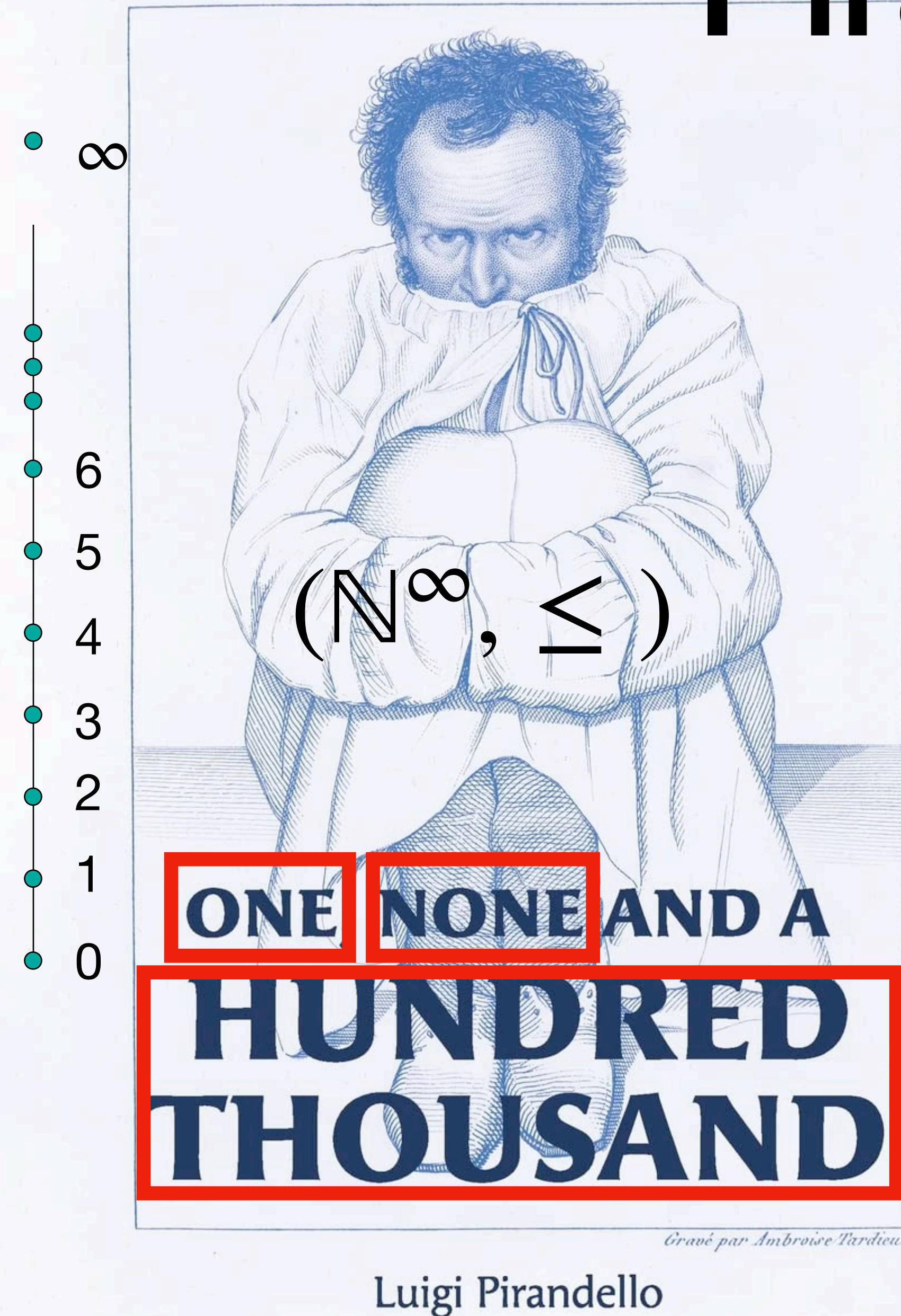
There is no grammatical distinction between [singular](#) and [plural](#), even in pronouns.

A 2012 documentary aired on the [Smithsonian Channel](#) reported that a school had been opened for the Pirahã community where they learn Portuguese and mathematics. As a consequence, observations involving concepts like the notion of quantity (which has a singular treatment in Pirahã language) became impossible, because of the influence of the new knowledge on the results.^[18]

[\[improper synthesis?\]](#)

Pirandello domain

Pirandello domain



$$\gamma(\top) = \infty$$

$$\gamma(1) = 1$$

$$\gamma(\perp) = 0$$

$$\alpha(0) = \perp$$

$$\alpha(1) = 1$$

$$\alpha(n) = \top \text{ otherwise}$$

$$(\mathbb{N}^\infty, \leq) \begin{array}{c} \xleftarrow{\gamma} \\ \xrightarrow{\alpha} \end{array} (A_{\mathbb{LP}}, \sqsubseteq)$$

$$\top \uparrow 1 \uparrow \perp$$

Exercise: sound abstractions?

$$\gamma(\top) = \infty$$

$$\gamma(1) = 1$$

$$\gamma(\perp) = 0$$

$+\#$	$\perp$	1	$\top$
$\perp$	$\top$	(commutative)	
1	1	1	
$\top$	$\top$	$\top$	$\top$

$+\#$	$\perp$	1	$\top$
$\perp$	1	(commutative)	
1	1	$\top$	
$\top$	$\top$	$\top$	$\top$

$+\#$	$\perp$	1	$\top$
$\perp$	$\perp$	(commutative)	
1	1	$\top$	
$\top$	$\top$	$\top$	$\top$

sound? BCA?

sound? BCA?

sound? BCA?

Exercise: BCA

$$\gamma(\top) = \infty$$

$$\gamma(1) = 1$$

$$\gamma(\perp) = 0$$

$\times^\#$	$\perp$	1	$\top$
$\perp$		(commutative)	
1			
$\top$			



Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation

Antoine Miné

► To cite this version:

Antoine Miné. Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation. Foundations and Trends in Programming Languages, Now Publishers, 2017, 4 (3-4), pp.120-372. 10.1561/25000000034 . hal-01657536

HAL Id: hal-01657536

<https://hal.sorbonne-universite.fr/hal-01657536>

Submitted on 1 May 2018

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers. L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Chapter 4: Non-relational abstract domains

Constant domains

Constant domain

a simple static analysis is *constant propagation*:
it detects if some variables or expressions take only a
single, fixed value during all executions and enables
compiler optimization

the constant domain finds application in such cases

Constant domain

$$(\wp(\mathbb{V}), \subseteq)$$

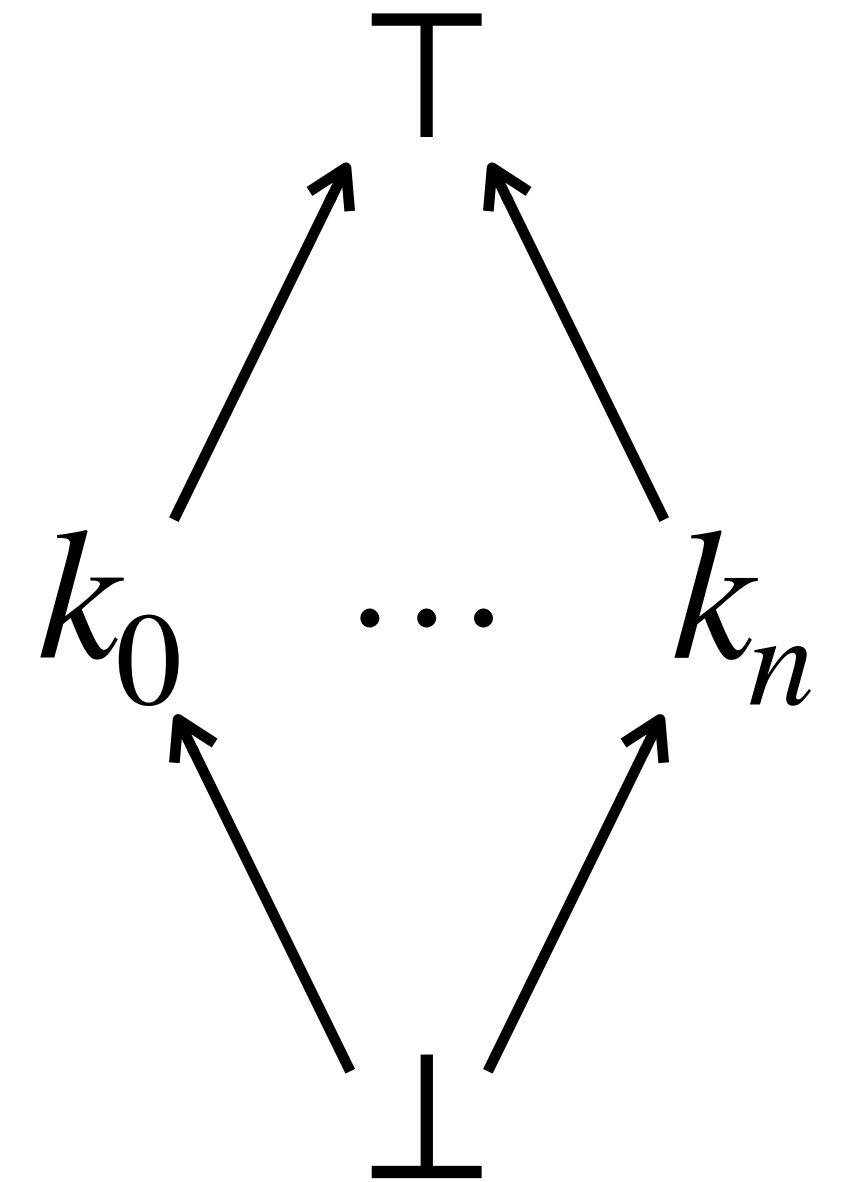
$$\gamma(\top) = \mathbb{V}$$

$$\gamma(k_0) = \{k_0\}$$

...

$$\gamma(k_n) = \{k_n\}$$

$$\gamma(\perp) = \emptyset$$



$$(\wp(\mathbb{V}), \subseteq) \begin{array}{c} \xleftarrow{\gamma} \\ \xrightarrow{\alpha} \end{array} (A_{\mathbb{V}}, \sqsubseteq)$$

Constant domain

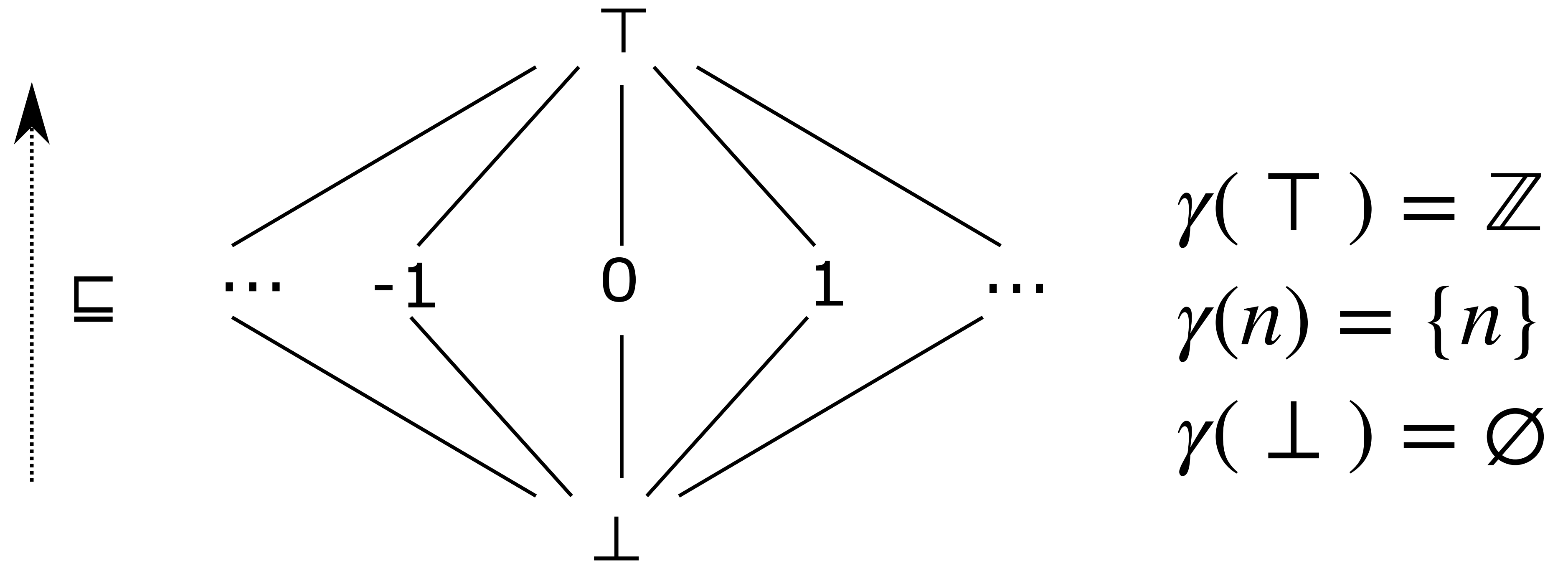


Figure 4.4: Hasse diagram for the constant domain.

$$(\wp(\mathbb{Z}), \subseteq) \begin{array}{c} \xleftarrow{\gamma} \\ \xrightarrow{\alpha} \end{array} (A_{\mathbb{Z}}, \sqsubseteq)$$

Exercise: sound abstraction

$$\gamma(\top) = \mathbb{Z}$$

$$\gamma(n) = \{n\}$$

$$\gamma(\perp) = \emptyset$$

$+\#$	$\perp$	m	$\top$
$\perp$	$\perp$	(commutative)	
n	$\perp$	$n + m$	
$\top$	$\perp$	$\top$	$\top$

BCA?

Exercise: sound abstraction

$$\gamma(\top) = \mathbb{Z}$$

$$\gamma(n) = \{n\}$$

$$\gamma(\perp) = \emptyset$$

$\times^\#$	$\perp$	m	$\top$
$\perp$	$\perp$	(commutative)	
n	$\perp$	$n \times m$	
$\top$	$\perp$	$\top$	$\top$

BCA?

Example

Example 4.2 (Constant analysis). *Consider the following very simple program:*

```
 $X \leftarrow 0; Y \leftarrow 10;$   
while  $X < 100$  do  
     $Y \leftarrow Y - 3;$   
     $X \leftarrow X + Y;$   $\ell^1$  which values for  $X$  and  $Y$ ?  
     $Y \leftarrow Y + 3$   
done
```

Example

Example 4.2 (Constant analysis). *Consider the following very simple program:*

	X	Y	
	$\top$	$\top$	$X \leftarrow 0; Y \leftarrow 10;$
	$\top$	10	while $X < 100$ do
	$\top$	10	$Y \leftarrow Y - 3;$
	$\top$	7	$X \leftarrow X + Y;$ ℓ_1
ℓ_1	$\top$	7	$Y \leftarrow Y + 3$
	$\top$	10	done $\top$ 10

which values for X and Y ?

X don't know, but $Y = 7$

An analysis with the non-relational constant domain in denotational style, following Sect. 3.5, will iterate the loop twice. At the first loop iteration, at program point ℓ_1 , we would get $X = 10, Y = 7$. During the second iteration, we join at the loop head abstract states where X can have two different values: 0 and 10, hence X is not constant. We get at program point ℓ_1 that $X = \top_b^\sharp, Y = 7$. The result is stable after the second iteration, so that $X = \top_b^\sharp, Y = 7$ is a program invariant at ℓ_1 : we have proved that Y equals 7 for every execution whenever it reaches ℓ_1 .

While this result may seem straightforward, we point out that Y varies during the execution of the program: it has a different constant value at different program points,

Exercise

Define the adjoint abstraction map $\alpha: \wp(\mathbb{Z}) \rightarrow A_{\mathbb{Z}}$

$$\gamma(\top) = \mathbb{Z}$$

$$\gamma(n) = \{n\}$$

$$\gamma(\perp) = \emptyset$$

$$\alpha = \begin{cases} \perp & X = \emptyset \\ n & \text{else } X = \{n\} \\ \top & \text{otherwise} \end{cases}$$

Constant set domain

the constant set domain generalizes the previous domain
by tracking several values instead of one

Constant set domain

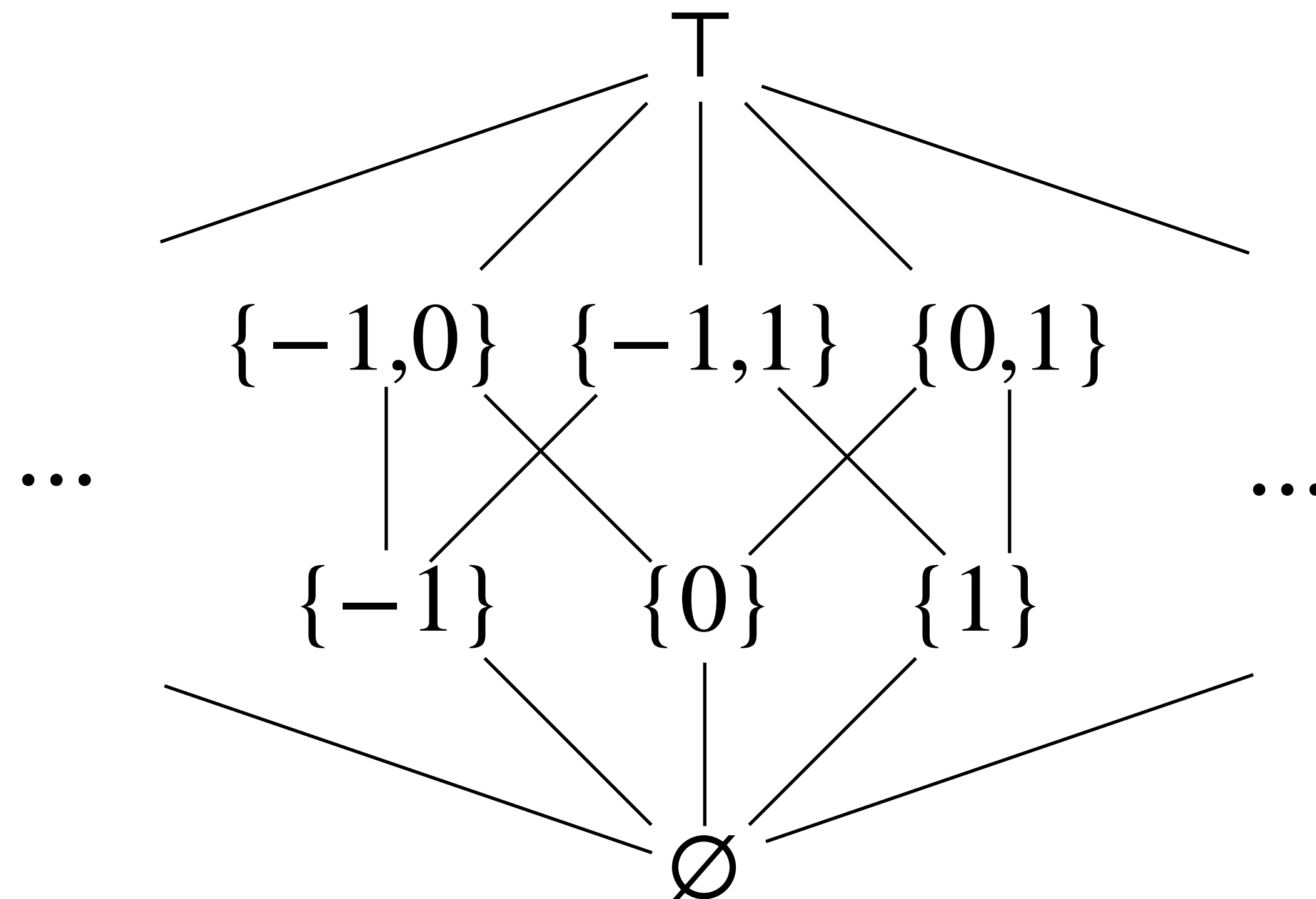
$$(\wp(\mathbb{V}), \subseteq)$$

$$\begin{aligned} \gamma(\top) &= \mathbb{V} \\ \gamma(X^\#) &= X^\# \quad \text{if } X^\# \subseteq \wp_k(K) \end{aligned}$$

$$\begin{array}{c} \top \\ \uparrow \\ \wp_k(K) \end{array}$$

$$(\wp(\mathbb{V}), \subseteq) \begin{array}{c} \xleftarrow{\gamma} \\ \xrightarrow{\alpha} \end{array} (A_{|k|}, \sqsubseteq)$$

Constant set domain



$$(\wp(\mathbb{Z}), \subseteq) \begin{array}{c} \xleftarrow{\gamma} \\ \xrightarrow{\alpha} \end{array} (A_{|2|}, \sqsubseteq)$$

Example

Example 4.2 (Constant analysis). *Consider the following very simple program:*

	<i>X</i>	<i>Y</i>	
	T	T	$X \leftarrow 0; Y \leftarrow 10;$
ℓ_1	T	10	while $X < 100$ do
	T	10	$Y \leftarrow Y - 3;$
	T	7	$X \leftarrow X + Y;$
	T	7	$Y \leftarrow Y + 3$
	T	10	done
	T	10	

which values for X and Y ?

X don't know, but $Y = 7$

Sound abstraction of sum

$$\begin{aligned} X^\# +^\# Y^\# &= \emptyset && \text{if } X^\# = \emptyset \text{ or } Y^\# = \emptyset \\ \text{BCA?} \quad &= \top && \text{else if } X^\# = \top \text{ or } Y^\# = \top \\ \quad \checkmark &= \top && \text{else if } |\{x + y \mid x \in X^\#, y \in Y^\#\}| > k \\ &= \{x + y \mid x \in X^\#, y \in Y^\#\} && \text{otherwise} \end{aligned}$$

Exercise

Define the BCA for product in $A_{|k|}$

$$\begin{aligned}
 X^\# \times Y^\# &= \emptyset \text{ if } X^\# = \emptyset \text{ or } Y^\# = \emptyset \\
 &= \{0\} \text{ if } X^\# = \{0\} \text{ or } Y^\# = \{0\} \\
 &= T \text{ if } X^\# = T \text{ or } Y^\# = T \text{ (} X^\# \neq \{0\}, Y^\# \neq \{0\} \text{)} \\
 &= T \text{ if } |\{x \cdot y \mid x \in X^\#, y \in Y^\#\}| > k \\
 &= \{x \cdot y \mid x \in X^\#, y \in Y^\#\} \text{ otherwise}
 \end{aligned}$$

Exercise

Define the adjoint abstraction map $\alpha: \wp(\mathbb{Z}) \rightarrow A_{|2|}$

$$\alpha = \begin{cases} S & |s| \leq 2 \\ T & |s| > 2 \end{cases}$$

Interval domain

4.5 The Interval Domain

We already mentioned interval abstractions several times in the previous chapters. Based on interval arithmetic, introduced by Moore [1966] for numeric analysis, and adapted to static analysis since the beginning of Abstract Interpretation by Cousot and Cousot [1976], it remains one of the most popular abstract domains because it is, as all non-relational domains, both simple and inexpensive, and yet it can express and infer valuable properties for program verification.

Intervals

$$(\{ [a, b] \mid a \in \mathbb{Z} \cup \{-\infty\}, b \in \mathbb{Z} \cup \{+\infty\}, a \leq b \} \cup \{\perp\}, \\ \subseteq, \sqcup, \cap, \perp, [-\infty, +\infty])$$

$$\gamma(\perp) = \emptyset$$

$$\gamma([a, b]) = \{x \in \mathbb{Z} \mid a \leq x \leq b\}$$

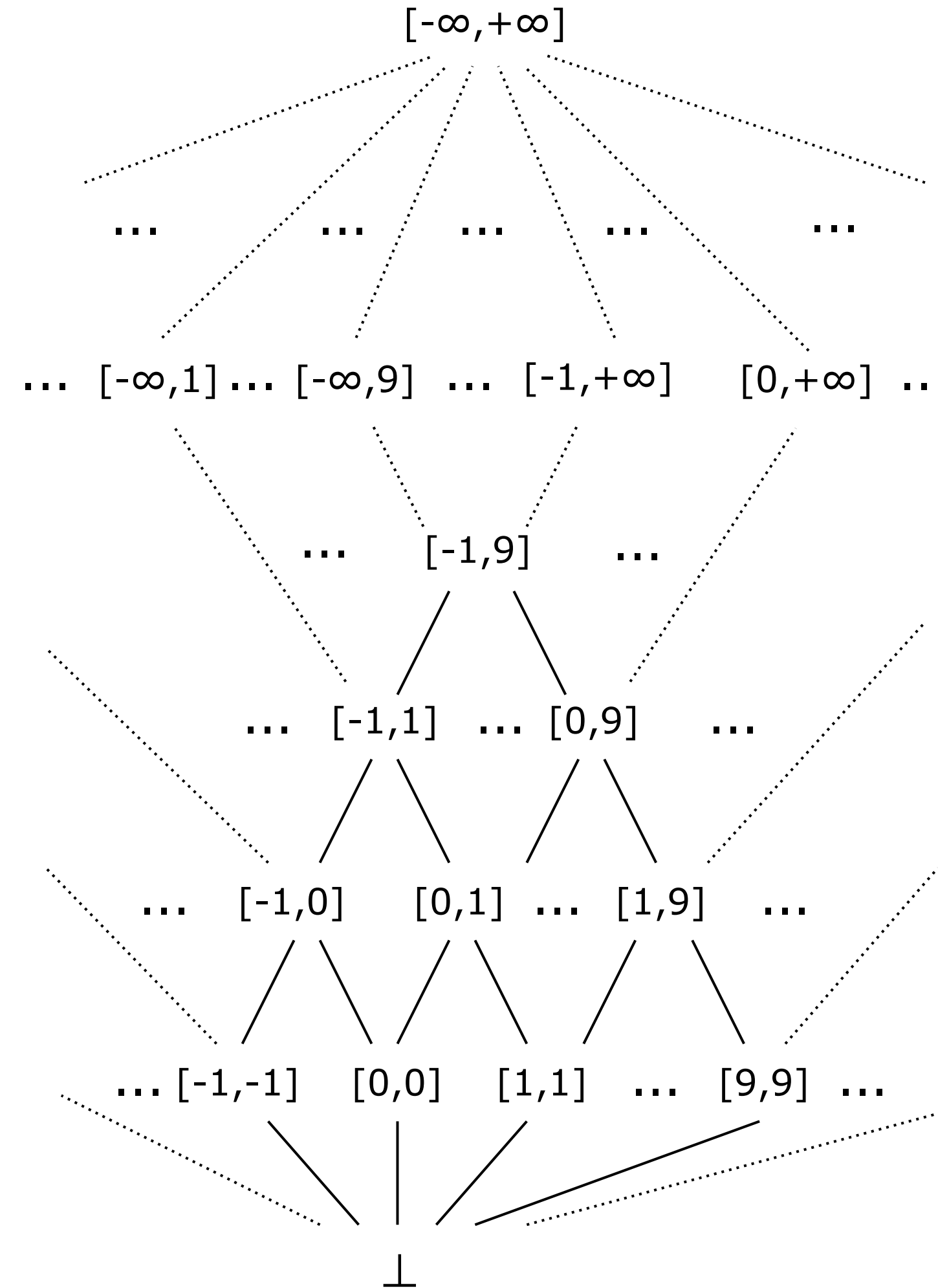
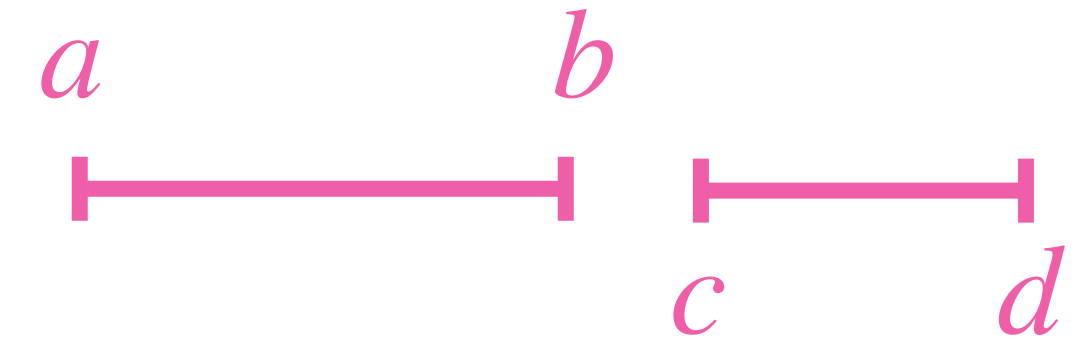
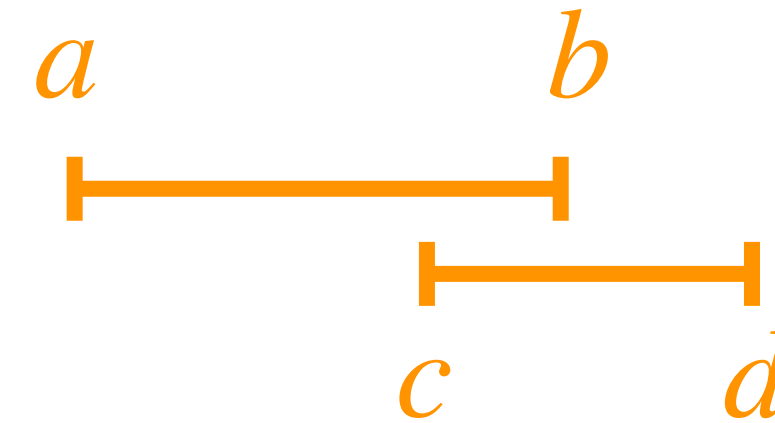
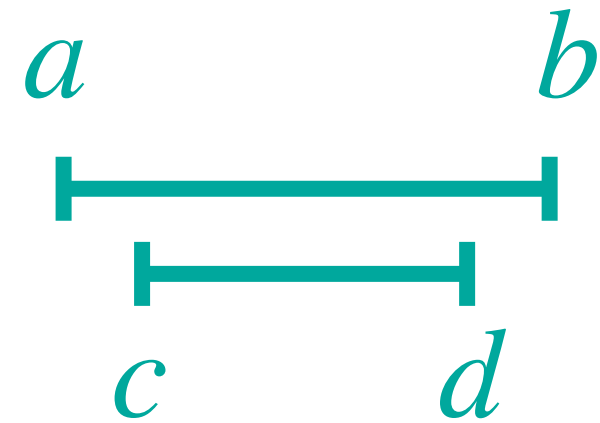


Figure 2.3: Hasse diagram for the interval lattice, with bounds extended to ∞ .

Intervals structure



$$\gamma(\perp) = \emptyset$$

$$\gamma([a, b]) = \{x \in \mathbb{Z} \mid a \leq x \leq b\}$$

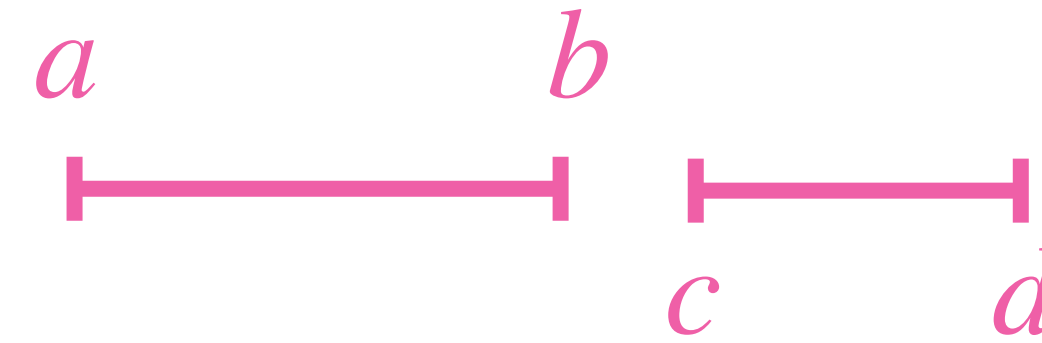
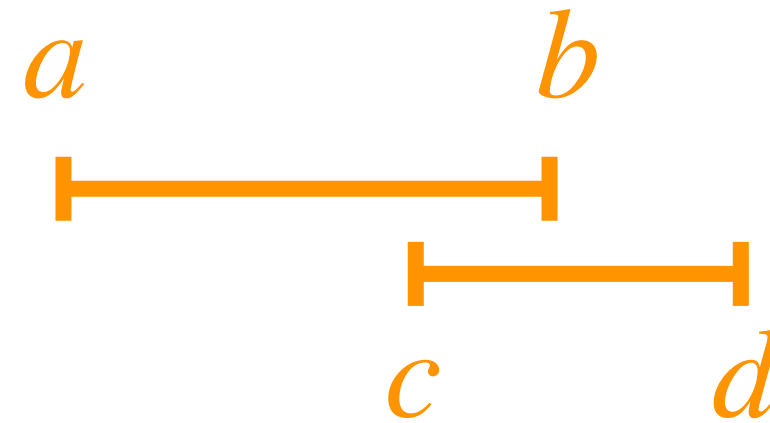
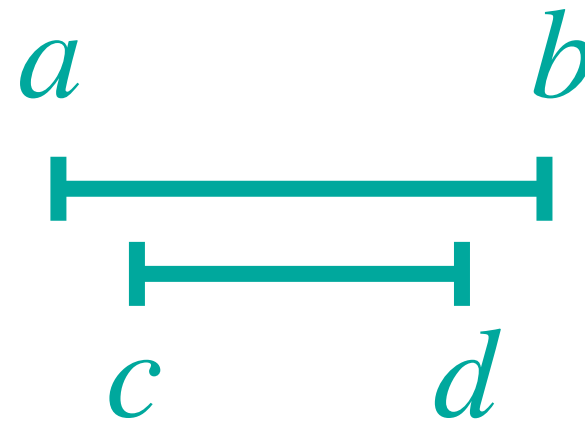
$$[a, b] \sqsubseteq [c, d] \iff (c \leq a) \wedge (b \leq d)$$

$$[a, b] \sqcup^\# [c, d] \triangleq [\min(a, c), \max(b, d)]$$

$$[a, b] \sqcap^\# [c, d] \triangleq [l, r] \quad \text{if } l = \max(a, c) \leq \min(b, d) = r$$

$\perp$ otherwise

BCAs for sum



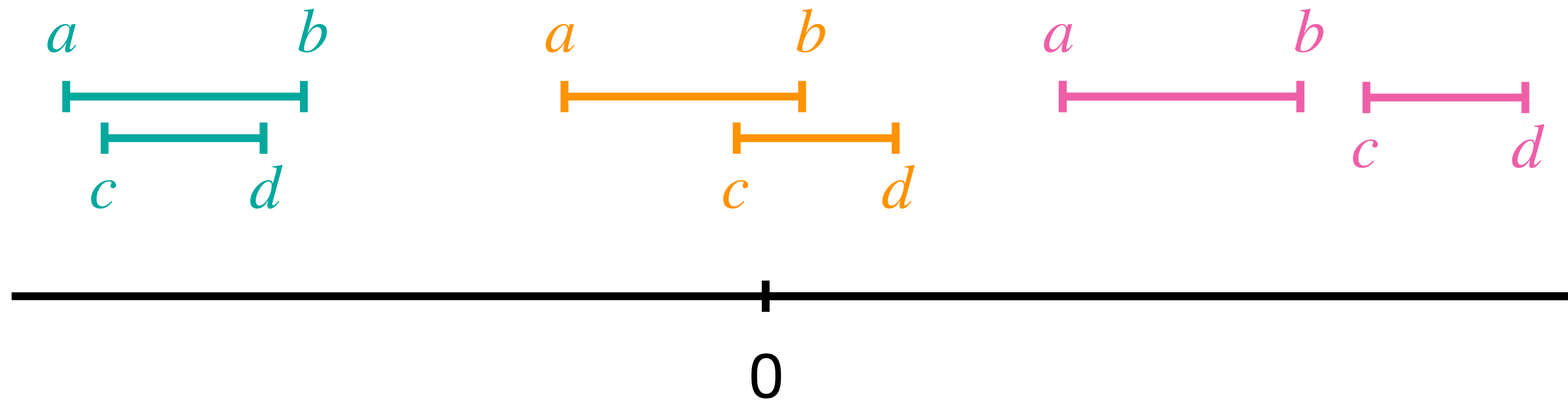
$$[a, b] +^{\#} [c, d] = [a + c, b + d]$$

$$[a, b] -^{\#} [c, d] = [a - d, b - c]$$

$$-^{\#} [a, b] = [-b, -a]$$

Exercise

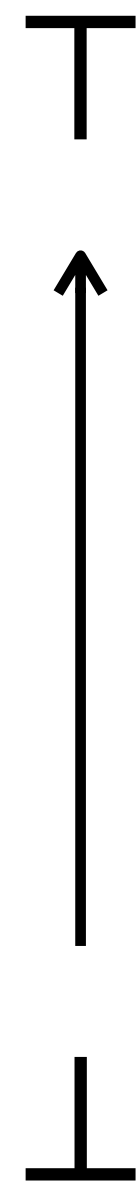
Define a sound abstraction for product in Int



$$[a, b] \times^{\#} [c, d] = [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]$$

Convergence of the analysis

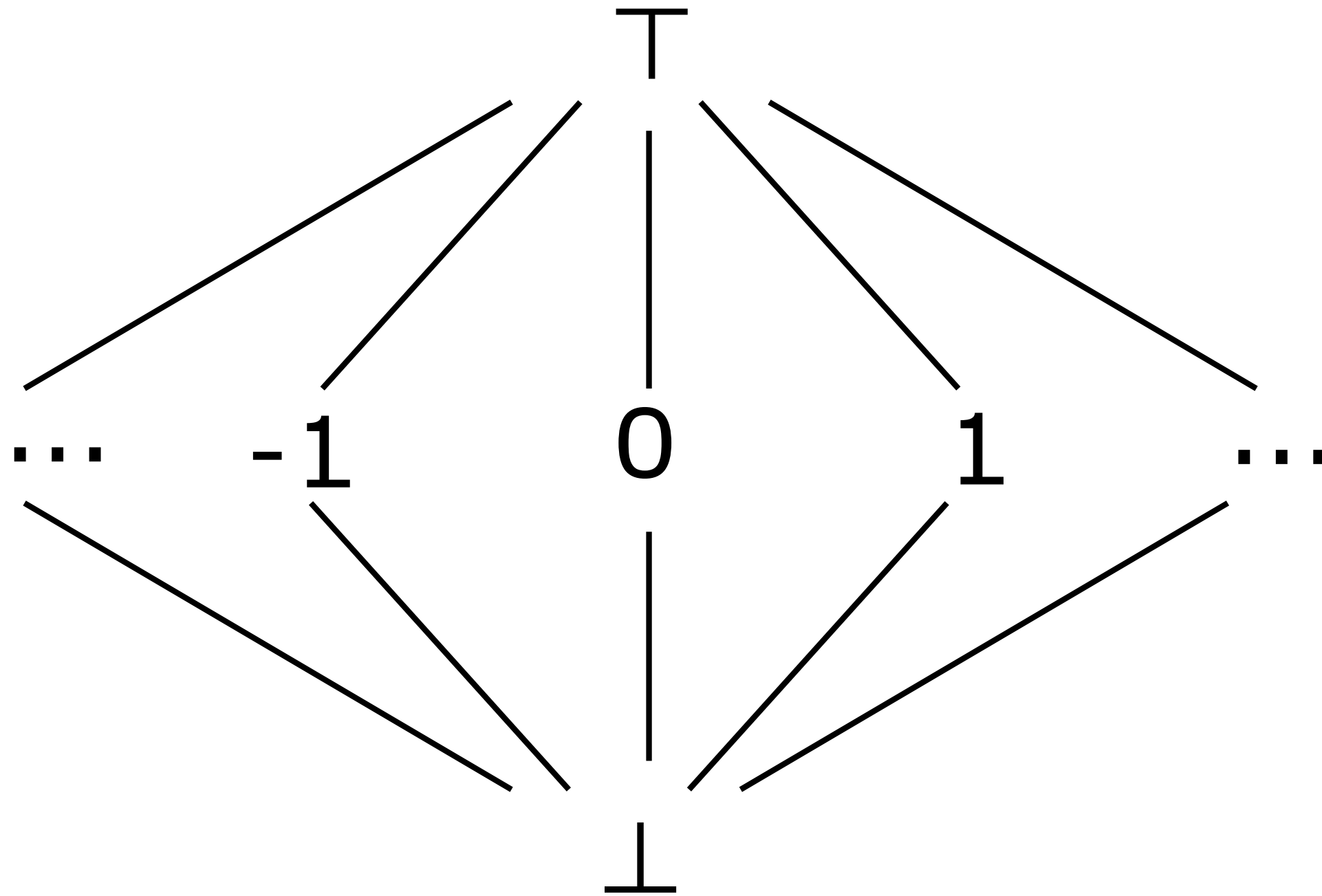
Find the (important) difference:



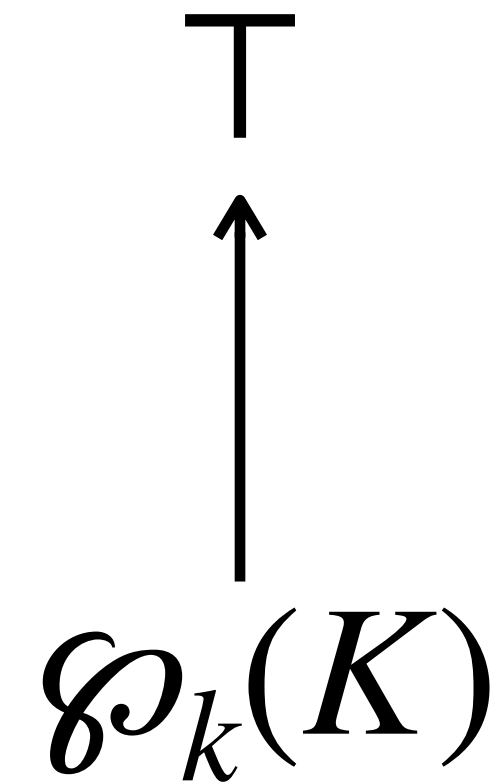
ACC!



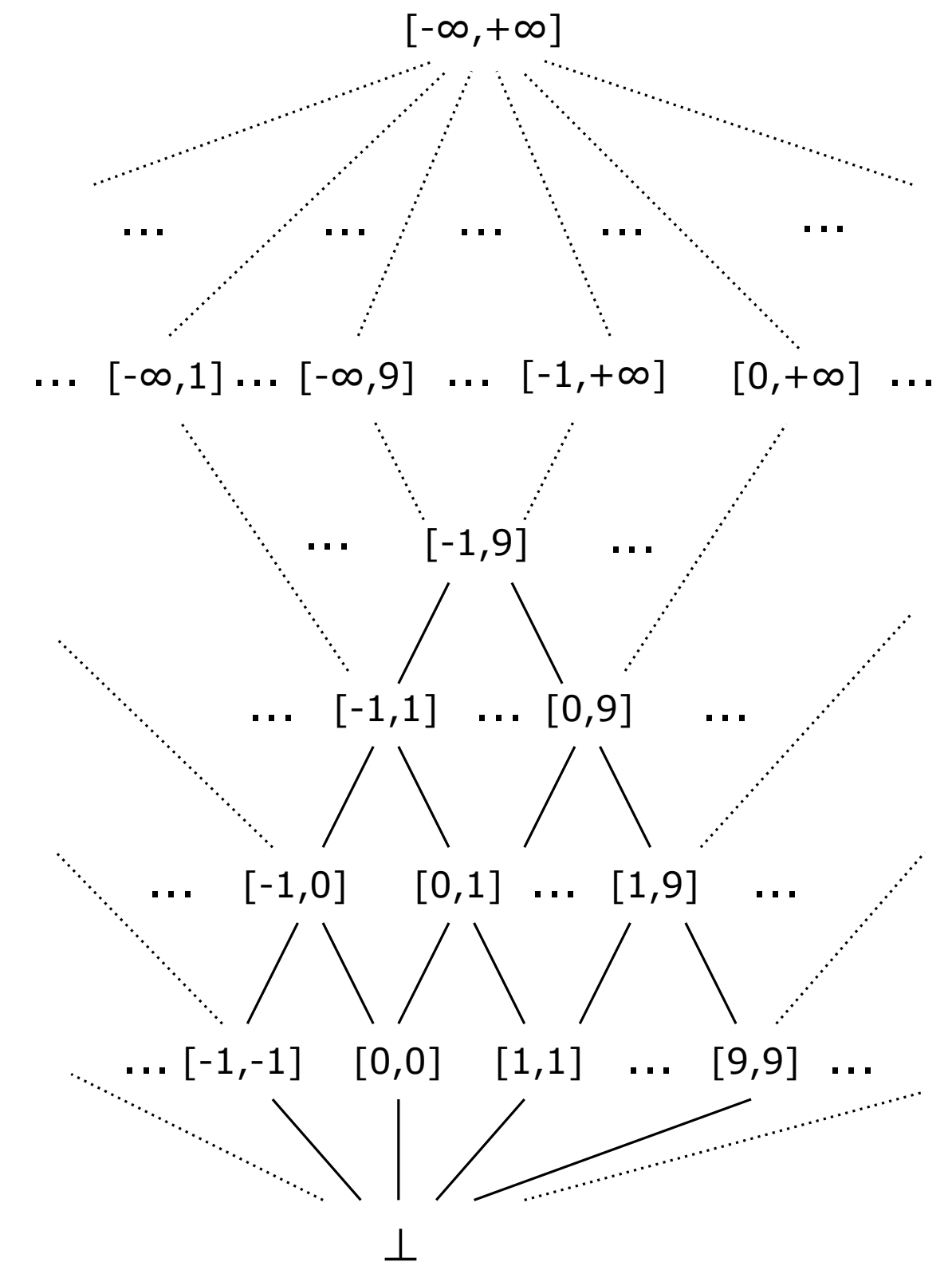
ACC!



ACC!



ACC!



not ACC!

Widening

Definition 2.17. A binary operator $\nabla : A \times A \rightarrow A$ is a widening operator in an abstract domain $(A, \sqsubseteq)$ if:

1. it computes upper bounds: $\forall x, y \in A: x \sqsubseteq x \nabla y$ and $y \sqsubseteq x \nabla y$
2. and it enforces convergence: for any sequence $(y^i)_{i \in \mathbb{N}}$ in A , the sequence $(x^i)_{i \in \mathbb{N}}$ computed as $x^0 \stackrel{\text{def}}{=} y^0, x^{i+1} \stackrel{\text{def}}{=} x^i \nabla y^{i+1}$ stabilizes in finite time. $\exists k \geq 0: x^{k+1} = x^k$. ■

Theorem 2.9. If f is a monotonic operator in a complete concrete lattice and g is a sound abstraction of f , then the following iteration:

$$\begin{array}{lcl} x^0 & \stackrel{\text{def}}{=} & \perp \\ x^{i+1} & \stackrel{\text{def}}{=} & x^i \nabla g(x^i) \end{array} \quad (2.6)$$

converges in finite time, and its limit x is a sound abstraction of the least fixpoint $\text{lfp } f$: $\text{lfp } f \leq \gamma(x)$. ■

Some widening

naive widening

$$x \nabla y \stackrel{\text{def}}{=} \begin{cases} x & \text{if } y \sqsubseteq x \\ \top & \text{otherwise} \end{cases}$$

The standard interval widening consists in replacing any unstable upper bound with $+\infty$, and any unstable lower bound with $-\infty$:

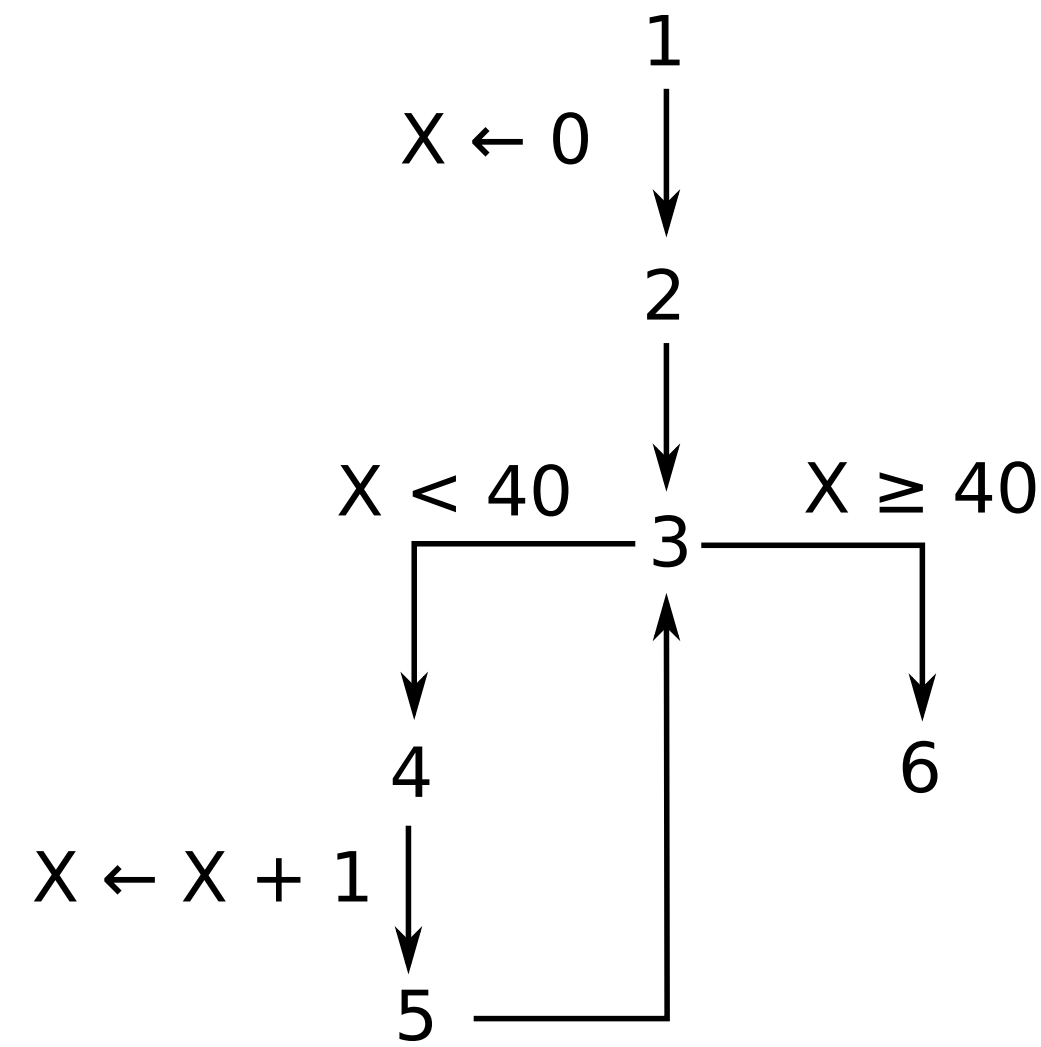
standard widening

$$[a, b] \nabla_b [c, d] \stackrel{\text{def}}{=} \left[\begin{cases} a & \text{if } a \leq c \\ -\infty & \text{otherwise} \end{cases}, \begin{cases} b & \text{if } b \geq d \\ +\infty & \text{otherwise} \end{cases} \right] \quad (4.7)$$

widening with
threshold

$$[a, b] \nabla_b^0 [c, d] \stackrel{\text{def}}{=} \left[\begin{cases} a & \text{if } a \leq c \\ 0 & \text{if } 0 \leq c < a \\ -\infty & \text{otherwise} \end{cases}, \begin{cases} b & \text{if } b \geq d \\ 0 & \text{if } 0 \geq d > b \\ +\infty & \text{otherwise} \end{cases} \right] \quad (4.12)$$

Example



(a)

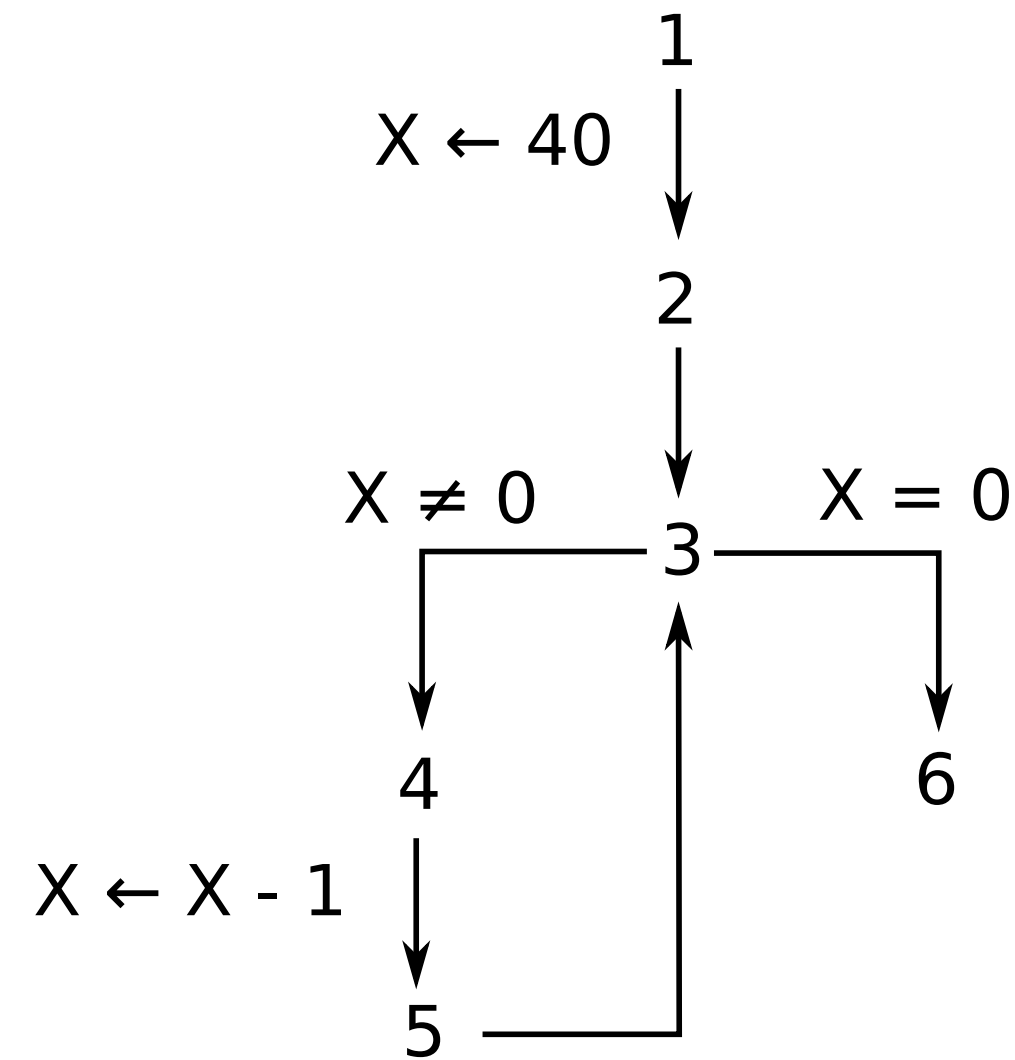
ℓ	$\mathcal{x}_{\ell}^{\#0}$	$\mathcal{x}_{\ell}^{\#4}$	$\mathcal{x}_{\ell}^{\#5}$	$\mathcal{x}_{\ell}^{\#7}$	$\mathcal{x}_{\ell}^{\#8}$
1	$\top$	$\top$	$\top$	$\top$	$\top$
2	$\perp$	0	0	0	0
3 ∇	$\perp$	0	$[0, +\infty]$	$[0, +\infty]$	$[0, +\infty]$
4	$\perp$	0	0	$[0, 39]$	$[0, 39]$
5	$\perp$	1	1	$[1, 40]$	$[1, 40]$
6	$\perp$	$\perp$	$\perp$	$[40, +\infty]$	$[40, +\infty]$

(b)

Figure 4.7: Interval analysis example: (a) control-flow graph of a simple loop incrementing X , and (b) abstract iterations.

$$[0,0] \nabla [1,1] = [0, +\infty]$$

Example



(a)

ℓ	intervals ∇	signs	intervals ∇^0
1	$\top$	$\top$	$\top$
2	40	> 0	40
3 ∇	$[-\infty, 40]$	≥ 0	$[0, 40]$
4	$[-\infty, 40]$	> 0	$[1, 40]$
5	$[-\infty, 39]$	≥ 0	$[0, 39]$
6	0	$= 0$	0

(b)

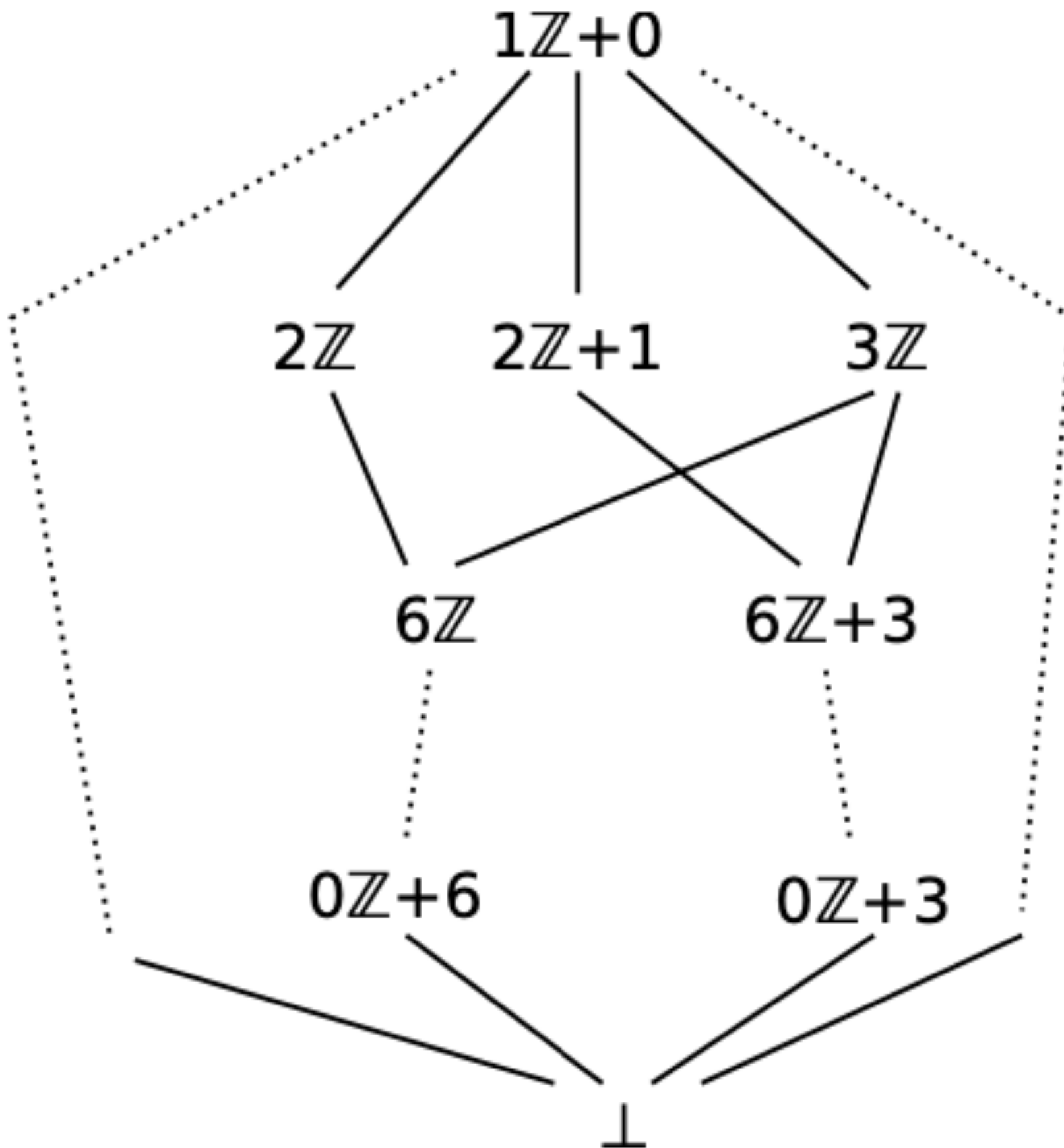
Figure 4.12: Analysis of a decremting loop using signs and intervals with various widenings.

Congruence domain

4.8 The Congruence Domain

The last non-relational abstract domain we present is a congruence domain, specific to integers, introduced by Granger [1989]. We thus assume $\mathbb{I} = \mathbb{Z}$, and infer value abstractions of the form $X \in a\mathbb{Z} + b$, meaning that X equals some multiple of a plus b . This domain is particularly useful to track pointer offsets in arrays and data-structures, prove alignment properties, or infer array access patterns.

Congruence domain



abstract elements of the form $a\mathbb{Z} + b$

(with $a \in \mathbb{N}$, $b \in \mathbb{Z}$, γ is not injective)

sometimes is assumed $0 \leq b < a$ (when $a \neq 0$)

(then γ becomes injective)

- the greatest element is $1\mathbb{Z} + 0$
- singletons $\{c\}$ are represented as $0\mathbb{Z} + c$
- the bottom element is explicitly added

Congruence domain structure

γ injective?

$$\gamma(3\mathbb{Z} + 2) = \gamma(3\mathbb{Z} + (2 + 3k)) \text{ for any } k \in \mathbb{Z}$$

$$\gamma(\perp) = \emptyset$$

$$\gamma(a\mathbb{Z} + b) = \{ak + b \mid k \in \mathbb{Z}\}$$

$$\alpha(X) = \sqcup_{x \in X}^{\#} 0\mathbb{Z} + x$$

Order structure. The lattice structure is depicted in Fig. 4.13. This lattice is based on some basic arithmetic results concerning divisors and so-called *cosets*. As we mentioned in Ex. 2.6, non-zero integers form a (non complete) lattice for divisibility: $(\mathbb{N}^*, |, \text{lcm}, \text{gcd})$, where $|$ is the “divides” partial order relation, lcm and gcd are the least common multiple and the greatest common divisor. We first have to extend this lattice to the case 0, which is useful in the domain to represent singletons:

- $y|y' \stackrel{\text{def}}{\iff} y \text{ divides } y', \text{ i.e., } \exists k \in \mathbb{N}: y' = ky, \text{ including } \forall y: y|0;$
- $x \equiv x' [y] \stackrel{\text{def}}{\iff} y|(x - x'), \text{ in particular, } x \equiv x' [0] \iff x = x';$
- $\vee$ is the lcm is extended with $y \vee 0 \stackrel{\text{def}}{=} 0 \vee y \stackrel{\text{def}}{=} 0;$
- $\wedge$ is the gcd extended with $y \wedge 0 \stackrel{\text{def}}{=} 0 \wedge y \stackrel{\text{def}}{=} y.$

Then, $(\mathbb{N}, |, \vee, \wedge, 1, 0)$ is a complete distributive lattice. We can define on top of this arithmetic lattice a complete lattice structure over $(\mathcal{B}^{\#}, \sqsubseteq_b^{\#}, \sqcup_b^{\#}, \sqcap_b^{\#}, \perp_b^{\#}, (1\mathbb{Z} + 0))$, where:

- $(a\mathbb{Z} + b) \sqsubseteq_b^{\#} (a'\mathbb{Z} + b') \stackrel{\text{def}}{\iff} a'|a \text{ and } b \equiv b' [a']$
- $(a\mathbb{Z} + b) \sqcup_b^{\#} (a'\mathbb{Z} + b') \stackrel{\text{def}}{=} (a \wedge a' \wedge |b - b'|)\mathbb{Z} + b$
- $(a\mathbb{Z} + b) \sqcap_b^{\#} (a'\mathbb{Z} + b') \stackrel{\text{def}}{=} \begin{cases} (a \vee a')\mathbb{Z} + b'' & \text{if } b \equiv b' [a \wedge a'] \\ \perp_b^{\#} & \text{otherwise} \end{cases}$
where b'' is such that $b'' \equiv b [a \vee a'] \equiv b' [a \vee a']$, and is given by Bézout’s identity and can be computed using the extended Euclidean algorithm.

Although the domain has an infinite height, there is no infinite strictly increasing chain!

BCAs

$$-^{\#}(a\mathbb{Z} + b) = (a\mathbb{Z} + (-b))$$

$$(a\mathbb{Z} + b) +^{\#} (c\mathbb{Z} + d) = (\gcd(a, c)\mathbb{Z} + (b + d))$$

$$(a\mathbb{Z} + b) -^{\#} (c\mathbb{Z} + d) = (\gcd(a, c)\mathbb{Z} + (b - d))$$

$$(a\mathbb{Z} + b) \times^{\#} (c\mathbb{Z} + d) = (\gcd(ac, ad, bc)\mathbb{Z} + bd)$$

Example

Example 4.11 (Congruence analysis). *Consider the program:*

```
 $X \leftarrow 0; Y \leftarrow 2;$   
while  $X < 40$  do   inferred loop invariant:  
     $X \leftarrow X + 2;$   
    if  $X < 5$  then  $Y \leftarrow Y + 18$  endif;  
    if  $X > 8$  then  $Y \leftarrow Y - 30$  endif  
done
```

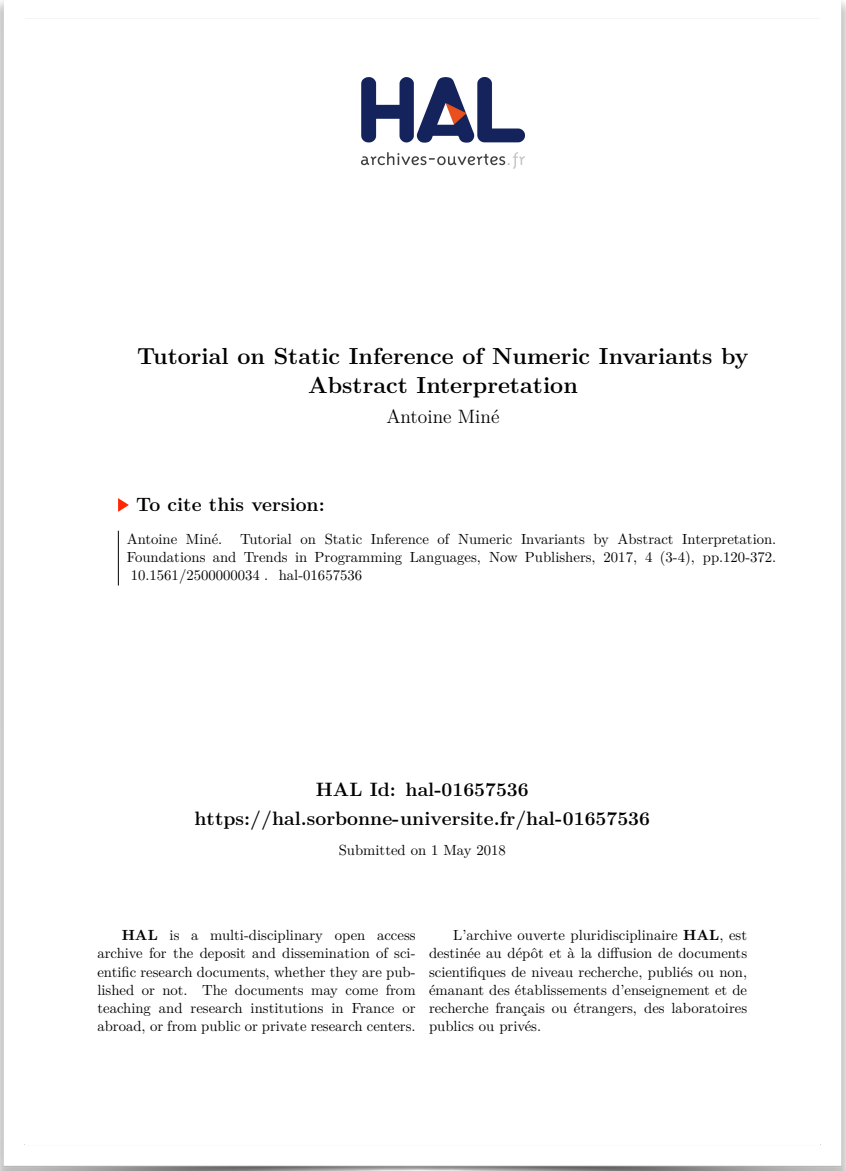
X is even

$$X \in (2\mathbb{Z} + 0)$$
$$Y \in (6\mathbb{Z} + 2)$$

gcd(18,30)
Y can be incremented by 18
or decremented by 30 at any iteration

Note that, except when an abstract value represents a constant, any test of the form $X < 40$ or $X > 8$ is abstracted as the identity.

Summary



signs	$0, (\geq 0), (> 0), (\leq 0), (< 0), (\neq 0)$	Sect. 4.2
constants	$c \in \mathbb{I}$	Sect. 4.3
constant sets	$C \in \mathcal{P}_{finite}(\mathbb{I})$	Sect. 4.4
intervals	$[a, b]$	Sect. 4.5
congruences	$a\mathbb{Z} + b$	Sect. 4.8



Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation

Antoine Miné

► To cite this version:

Antoine Miné. Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation. Foundations and Trends in Programming Languages, Now Publishers, 2017, 4 (3-4), pp.120-372. 10.1561/25000000034 . hal-01657536

HAL Id: hal-01657536

<https://hal.sorbonne-universite.fr/hal-01657536>

Submitted on 1 May 2018

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers. L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Overview of Chapter 5: Relational abstract domains

Convex Polyhedra domain

sets of numerical constraints of the form

$$c_1x + c_2y \leq c$$

(at most two variables per constraint, with coefficients)

is a **relational** domain, can be used
when **intervals are not expressive enough**

does not admit a best abstraction

Octagons domain

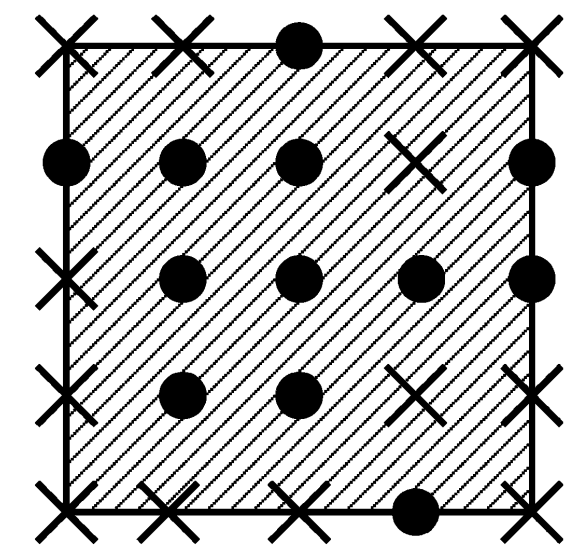
sets of numerical constraints of the form

$$\pm x \pm y \leq c$$

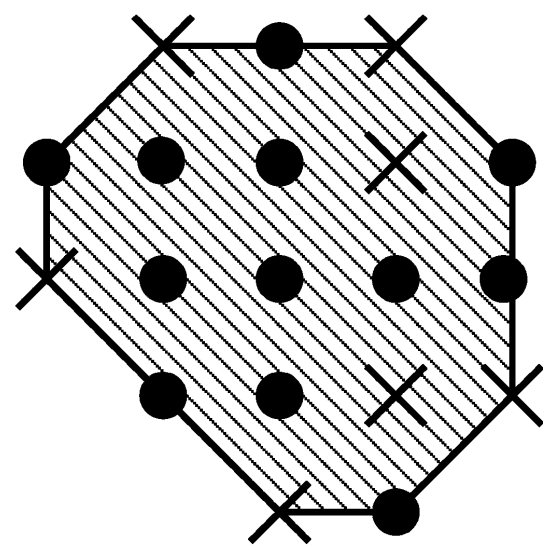
(at most two variables per constraint, with **unit** coefficients)

is a **relational** domain, can be used
when **intervals are not expressive enough**
and **polyhedra are more costly**
(allow trading precision for efficiency)

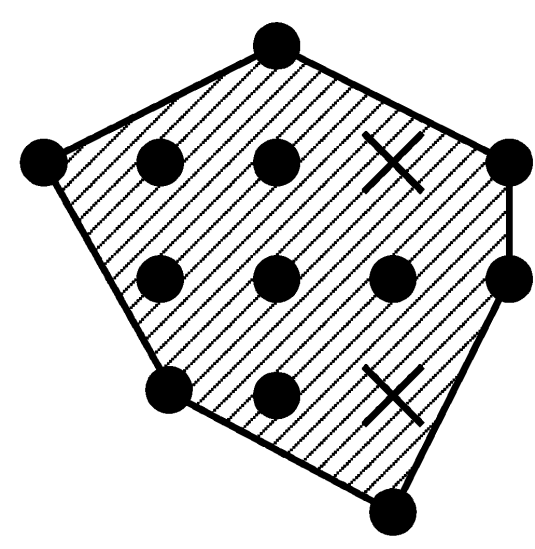
Summary



Intervals



Octagons



Polyhedra

domain	invariants	memory cost	time cost	Sect.
affine equalities	$\sum_i \alpha_i V_i = \beta$	$\mathcal{O}(\mathbb{V} ^2)$	$\mathcal{O}(\mathbb{V} ^3)$	5.2
zones	$V_j - V_i \leq c$	$\mathcal{O}(\mathbb{V} ^2)$	$\mathcal{O}(\mathbb{V} ^3)$	5.4
octagons	$\pm V_j \pm V_i \leq c$	$\mathcal{O}(\mathbb{V} ^2)$	$\mathcal{O}(\mathbb{V} ^3)$	5.4.5
template	$\sum_i \alpha_i V_i \geq \beta$	$\mathcal{O}(m)$	polynomial	5.5
polyhedra	$\sum_i \alpha_i V_i \geq \beta$	exponential	exponential	5.3

fixed coefficients

affine inequalities

Take home message

With abstract
domains, invariants are
inferred, not guessed

